

به نام خداوند جان و خرد

موضوع : آشنایی با مبانی میکروپایتون و اینترنت اشیا

تاریخ تهیه و تنظیم : آذر ماه ۱۴۰۴

دانشگاه آزاد اسلامی اصفهان (خوراسگان)



تهیه و تنظیم : مهندس یاسین سموعی

فهرست موضوعات:

- [میکروپایتون چیست ؟](#)
- [سخت افزار های مناسب میکروپایتون](#)
 - Raspberry pi ✓
 - Node Mcu ESP ✓
- [IDE مناسب میکروپایتون](#)
- [نصب خودکار میکرو پایتون](#)
- [نصب دستی میکروپایتون](#)
 - rasberry pico نصب روی ✓
 - ✓ آماده سازی ESP برای نصب (نصب esptool و درایور)
 - ✓ نصب روی ESP
- [معرفی شبیه ساز IOT](#)
- [مقدمه کد نویسی](#)
- [ذخیره سازی کد ها](#)
- [تعیین ورودی و خروجی ها](#)
- [استفاده از PWM](#)
- [راه اندازی ماژول های امنیتی](#)
 - ✓ ماژول رله
 - ✓ ماژول مگنت
 - ✓ ماژول PIR
 - ✓ ماژول التراسونیک
- [راه اندازی ماژول های نظارتی \(مانیتورینگ محیط \)](#)
 - ✓ ماژول DHT
 - ✓ سنسور فتوسل
 - ✓ ماژول MQX
 - ✓ ماژول رطوبت خاک
 - ✓ ماژول های neopixel
- [آشنایی با اتصال به WIFI](#)
- [معرفی منابع](#)



۱ میکروپایتون چیست؟

[MicroPython](#) یک پیاده‌سازی از زبان برنامه‌نویسی پایتون است که شامل زیرمجموعه کوچکی از کتابخانه استاندارد پایتون است و برای اجرا بر روی میکروکنترلرها و در محیط‌های محدود بهینه شده است.

علاوه بر پیاده‌سازی مجموعه‌ای از کتابخانه‌های هسته پایتون، MicroPython شامل ماژول‌هایی مانند «Machine» برای دسترسی به سخت‌افزار سطح پایین است.

۲ سخت افزار های مناسب میکروپایتون:

۱. سخت افزار raspberry pi :

خانواده Raspberry Pi Pico در حال حاضر از چهار برد تشکیل شده است.



Raspberry Pi Pico



Pico H

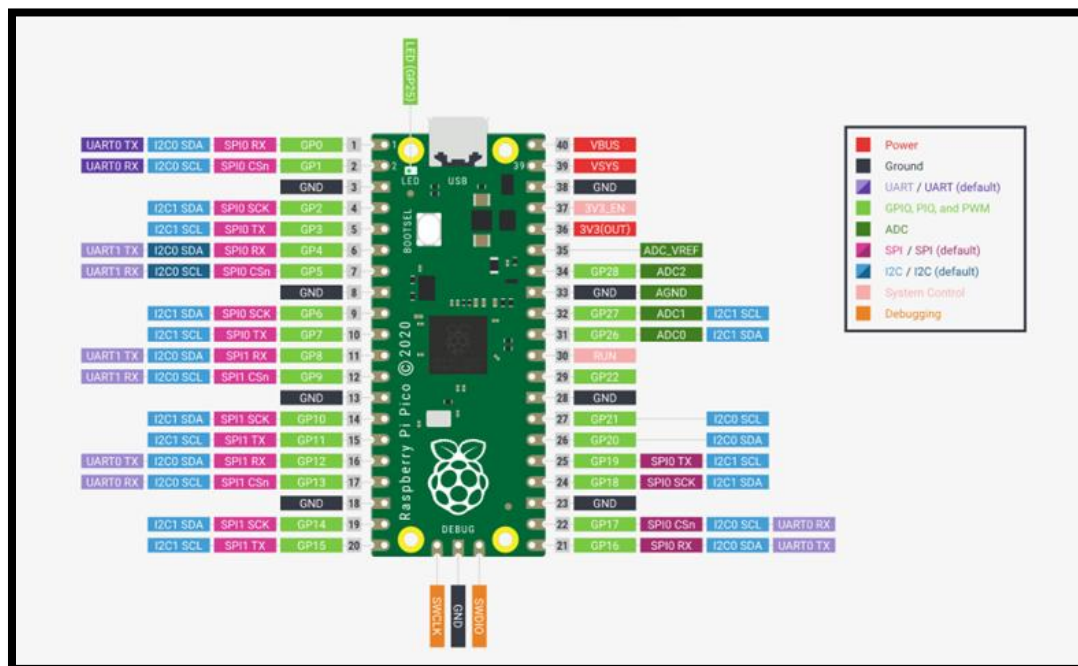


Pico W

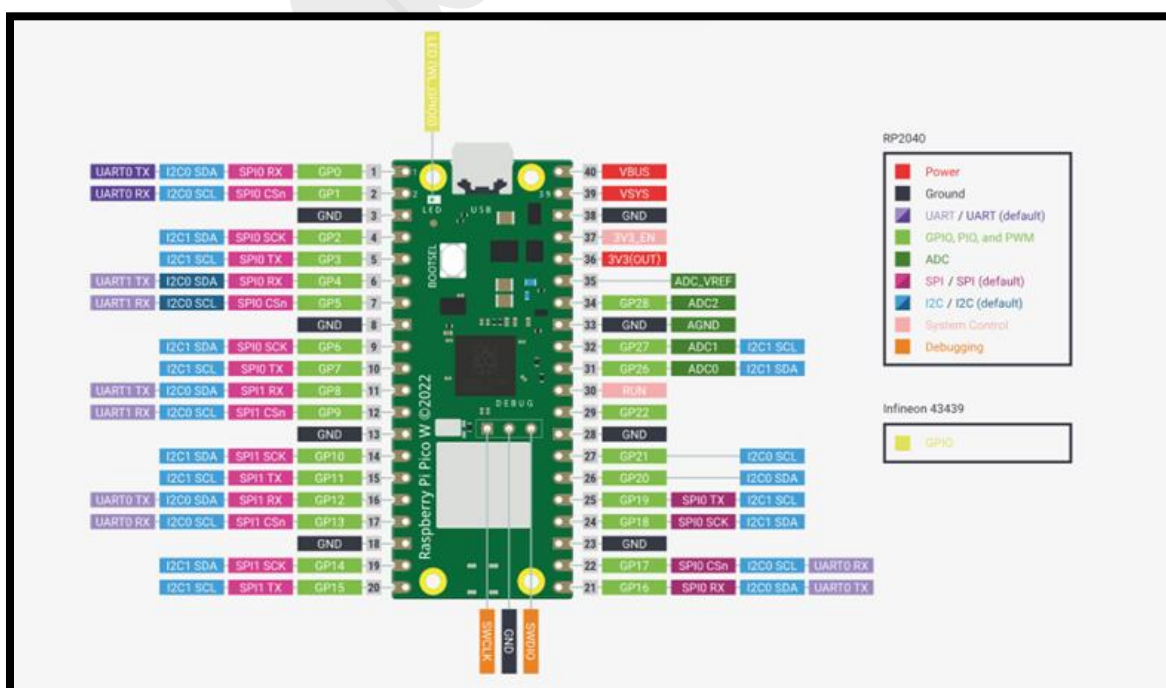


Pico WH

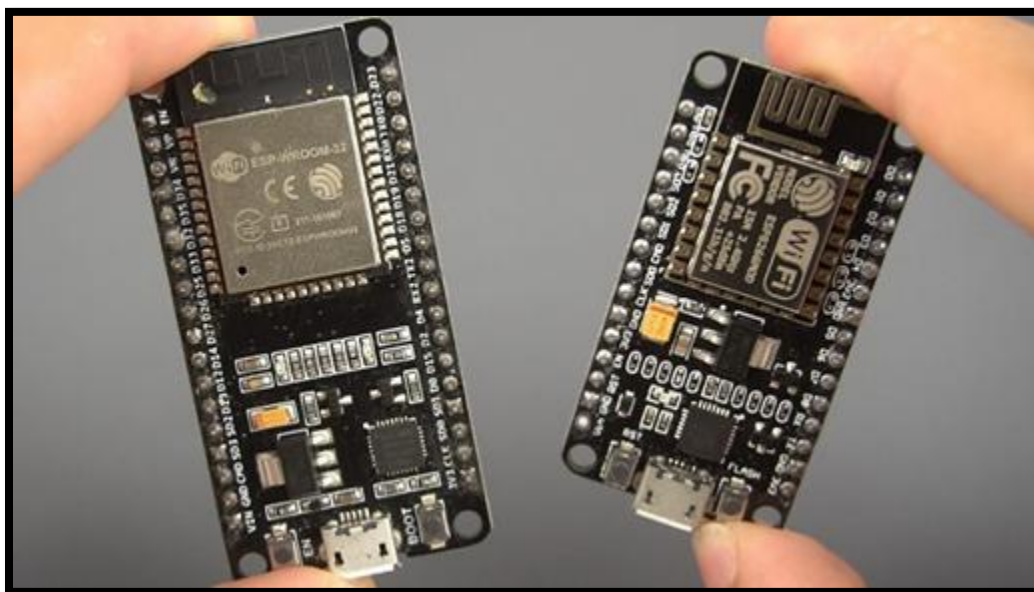
ساختار پایه های رزبری پیکو:



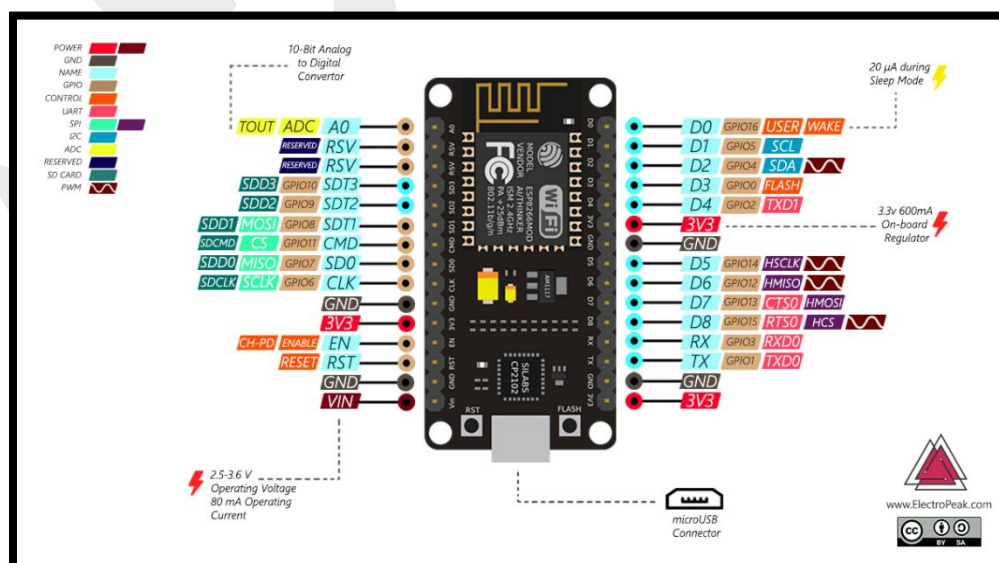
ساختار پایه های Raspberry Pi Pico W and Pico WH:



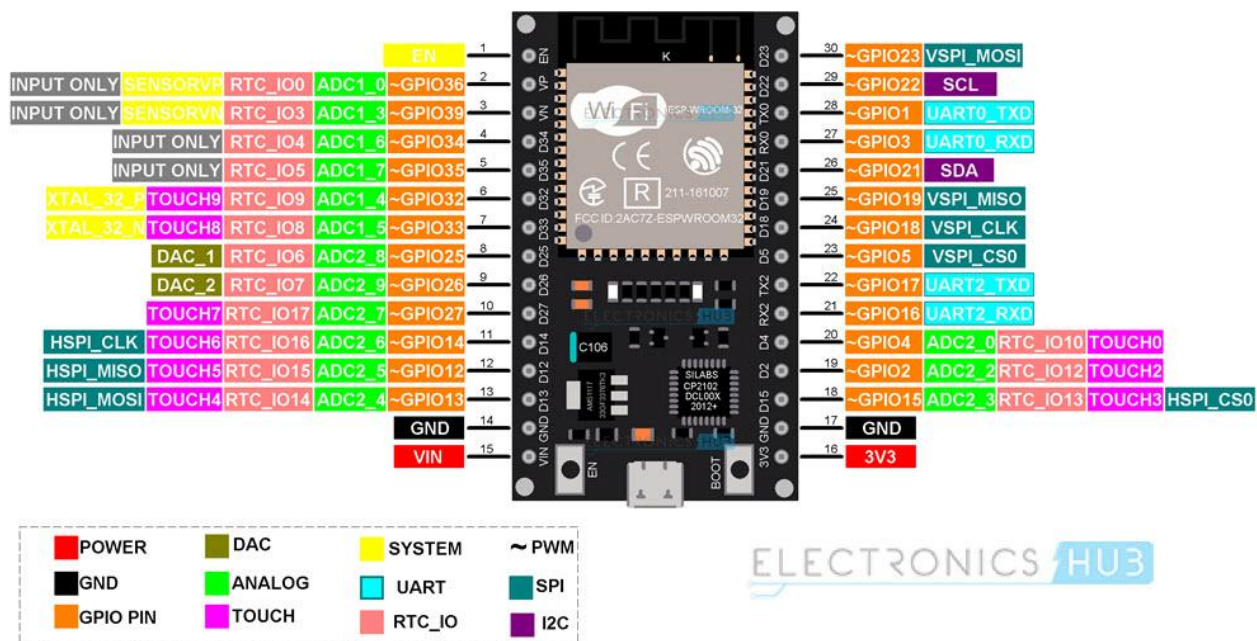
بورد های ESP در دو نوع ESP8266 و ESP32 هستند :



نمای پایه های ESP8266:



نمای پایه های ESP32:



۳ IDE مناسب میکروپایتون:

از Thonny یا VScode می توانیم برای توسعه میکروپایتون استفاده کنیم :

استفاده از Thonny:

برای این کار دو روش داریم یکی استفاده از pip و دیگری نصب از طریق آدرس سایت Thonny:

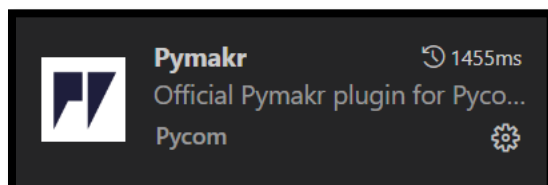
دستور نصب: `pip install thonny`

سایت نصب : <https://thonny.org>

در ادامه برای نصب میکروپایتون و اجرای کد ها از thonny استفاده می کنیم اما نگاهی هم به vscode به دلیل محبوبیت زیاد آن داریم.

استفاده از visual studio code:

برای استفاده از vs code ابتدا نیاز به نصب یک extension به نام pymakr داریم.

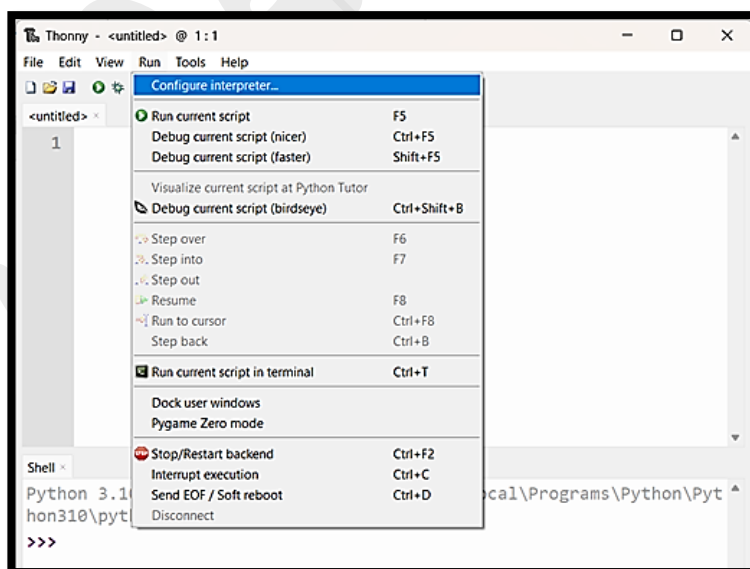


سپس از طریق این افزونه می توانیم دستگاه خود را متصل کنیم و کدهای خود را روی آن آپلود کنیم.

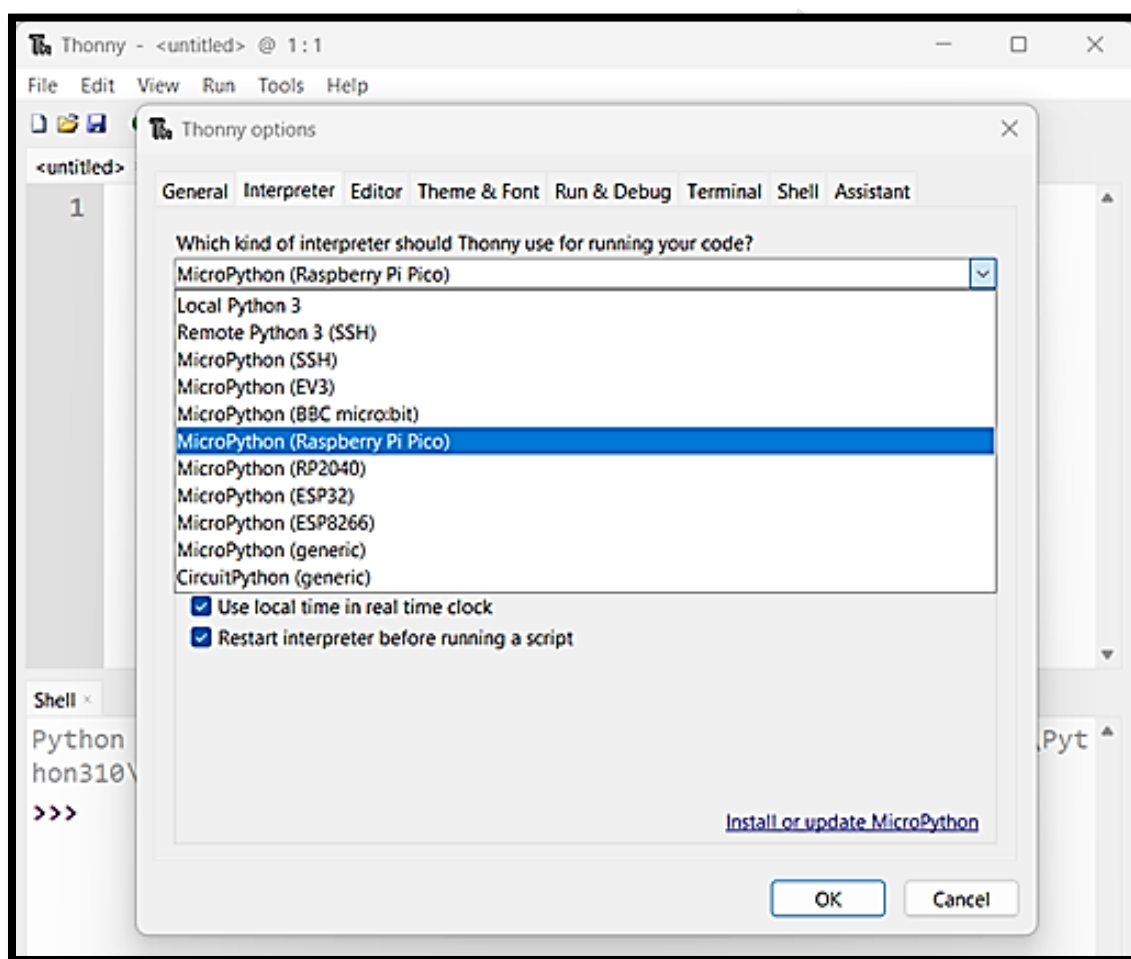
۴ نصب خودکار میکروپایتون:

برای نصب میکروپایتون به صورت خودکار روی هر کدام از دستگاه های معرفی شده ابتدا وارد thonny می شویم.

سپس در منو های بالا بر روی RUN>Configure Interpreter کلیک کنید.

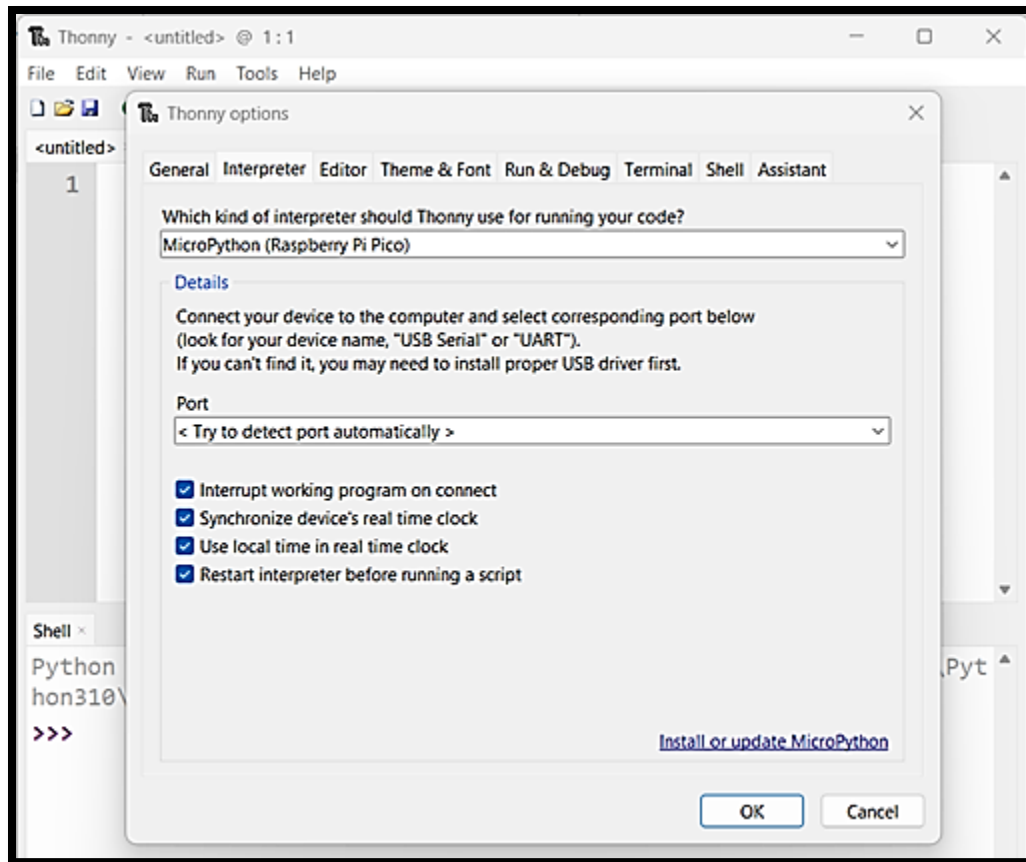


سپس در صفحه مربوطه بر روی انتخاب مکانی که می خواهید کد شما اجرا شود کلیک کرده و از گزینه های موجود بر روی نام دستگاه مد نظر مثلا (Raspberry pi pico) کلیک نمایید.

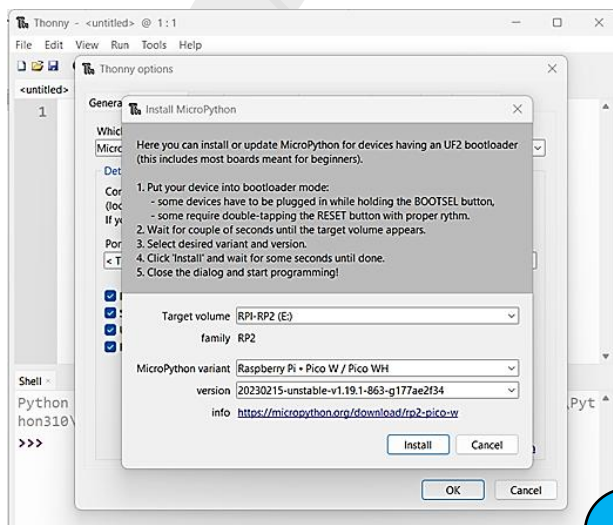


سپس بر روی بخش انتخاب پورت کلیک کرده و پورت مورد نظر را انتخاب کنید و در صورت عدم مشاهده پورت مورد نظر بر روی شناسایی اتوماتیک قرار دهید.

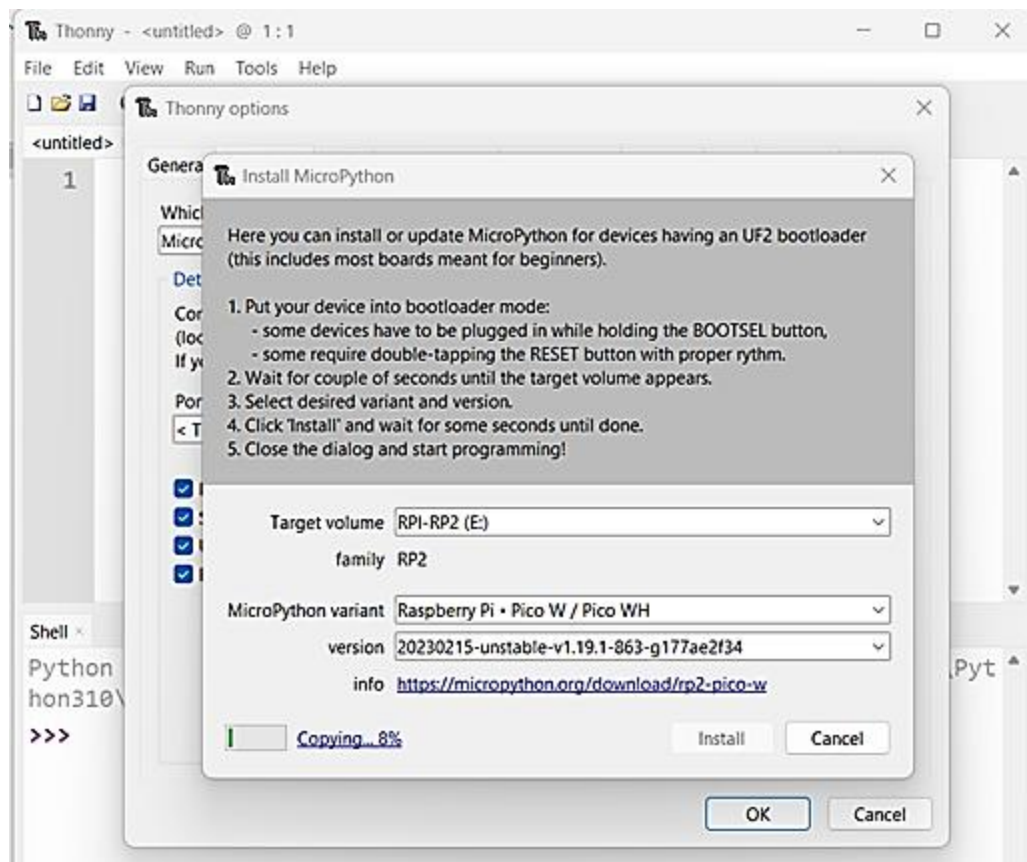




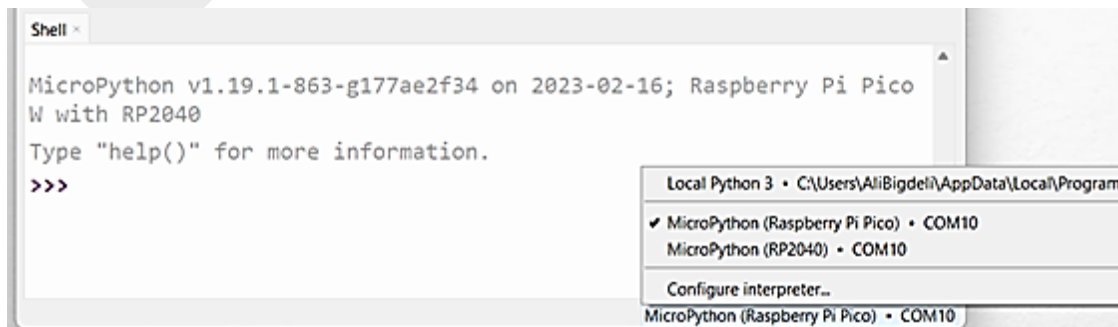
حالا بر روی گزینه install or update micropython کلیک نمایید.
در این بخش (نیاز به اینترنت خواهید داشت) با کلیک بر روی variant و
انتخاب مورد مناسب و در ادامه انتخاب نسخه پایتون مورد نظر فرم ها را پر
کنید.



در ادامه بر روی **install** کلیک نمایید. و مشاهده خواهید کرد که عملیات نصب و کپی آغاز خواهد شد.



تا زمان بالا اومدن مجدد بورد به صورت اتوماتیک صبر نمایید.
در صورتی که نصب موفق داشته باشید می توانید در بخش **interpreter** ها بورد مورد نظر را مشاهده نمایید.



برای نصب روی ESP نیز همین مراحل طی می شود.

فقط نکته مهم اینست که ESP برای اتصال به کامپیوتر از درایور **CP2102** استفاده می کند و نیاز به نصب ابزار **ESPTOOL** نیز داریم:

لینک دانلود درایور CP2102:

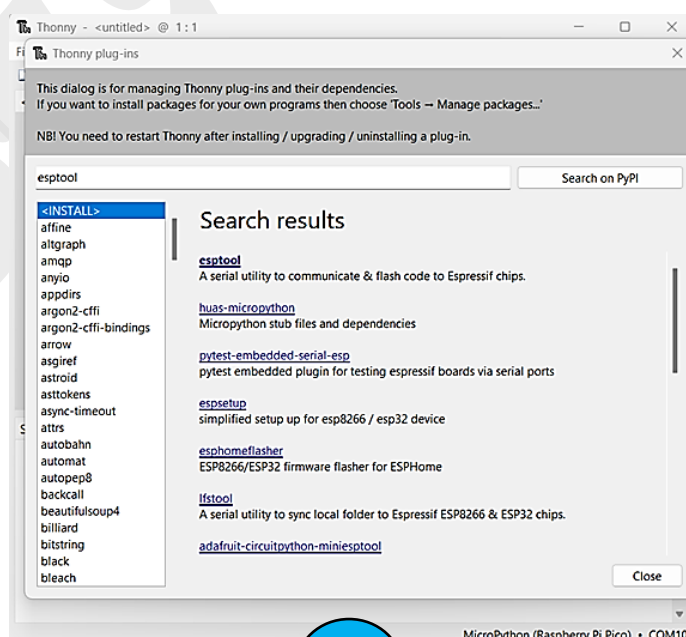
<https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers?tab=downloads>

کد نصب ESPTOOL:

```
pip install esptool
```

یا می توانیم نصب را از طریق خود **thonny** انجام دهیم:

در بخش **tools** به صفحه **manage plugins** وارد شوید. و در بخش جست و جو ابزار **esptool** را وارد کنید. پس از پیدا شدن، بر روی اون کلیک کرده و در مرحله بعدی گزینه نصب را زده و پس از نصب برنامه را بسته و مجدد باز نمایید.



۵ نصب دستی میکرو پایتون:

نصب روی رزبری :

برای نصب به صورت غیر خودکار کافی هستش که فایل مورد نظر را از داخل آدرس های زیر برای برد مناسب خود دانلود نمایید .

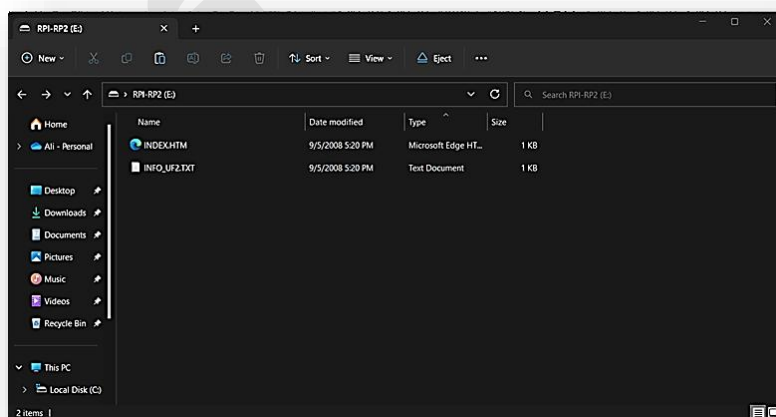
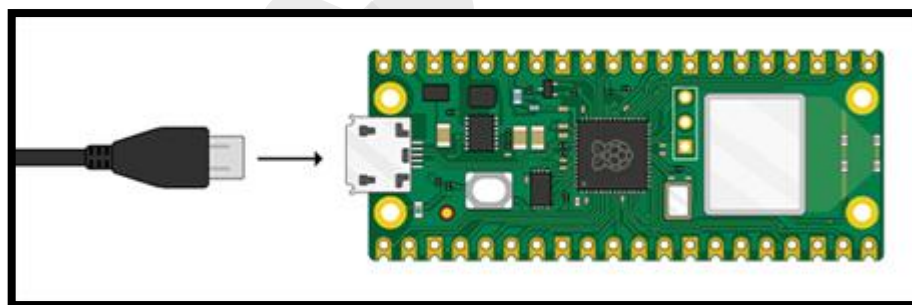
برد: PICO

<https://micropython.org/download/rp2-pico/>

برد: PICO W

<https://micropython.org/download/rp2-pico-w/>

سپس با نگه داشتن کلید سفید رنگ بر روی برد و اتصال مجدد به سیستم مورد نظر وارد درایو آماده شده شوید. که به شکل زیر خواهد بود.



در نهایت

پنجره روبه رو

برای شما باز

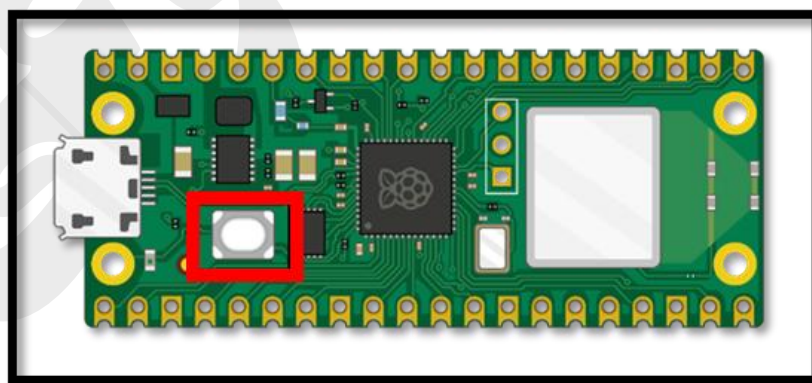
خواهد شد.

فایل مورد نظر را به داخل این پنجره Drag & Drop نمایید و یا Copy & Paste، سپس بورد restart خواهد شد و میکرو پایتون بر روی آن نصب خواهد شد.

نحوه Factory Reset رزبری :

برای اینکه بتوانید بورد خود را به طور کامل پاک کنید، البته در صورتی که به ایرادی با نصب با micropython برخورد کرده باشید. می توانید با دانلود فایل flash nuke از آدرس زیر اقدام نمایید.
<https://www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico.html#resetting-flash-memory>

سپس بورد را با نگهداری کلید به سیستم متصل نمایید. سپس فایل مربوطه را به داخل آن Drag & Drop و یا Copy & Paste نمایید. در نهایت تا پایان عملیات پاک سازی صبر نمایید.



نصب روی ESP:

ابتدا درایور نامبرده و esptool را نصب می کنیم.

حالا باید فایل میکروپایتون مربوطه را دانلود کنیم:

بورد ESP32:

<https://micropython.org/download/esp32/>

بورد ESP8266 :

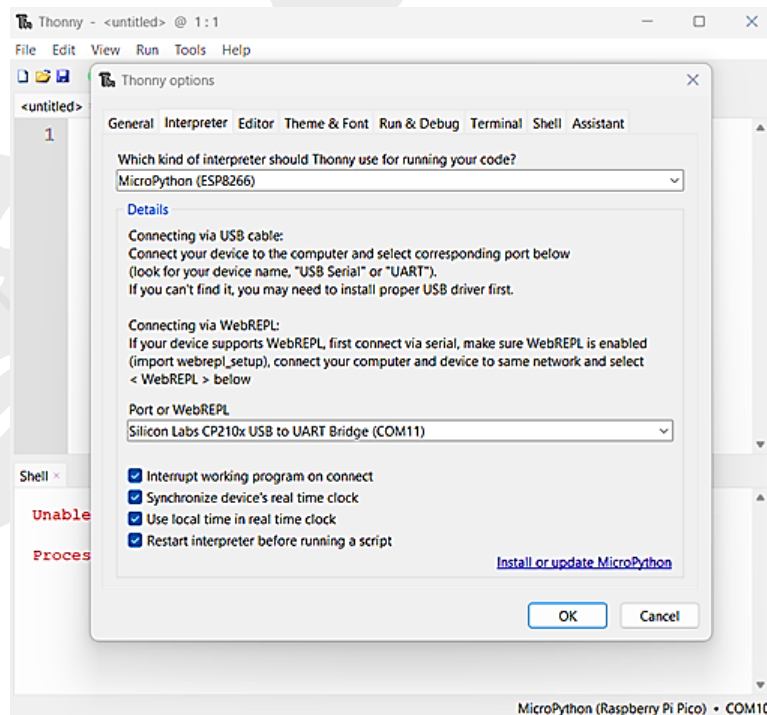
<https://micropython.org/download/esp8266/>

حالا بورد را با حالت flashmode به سیستم متصل کنیم

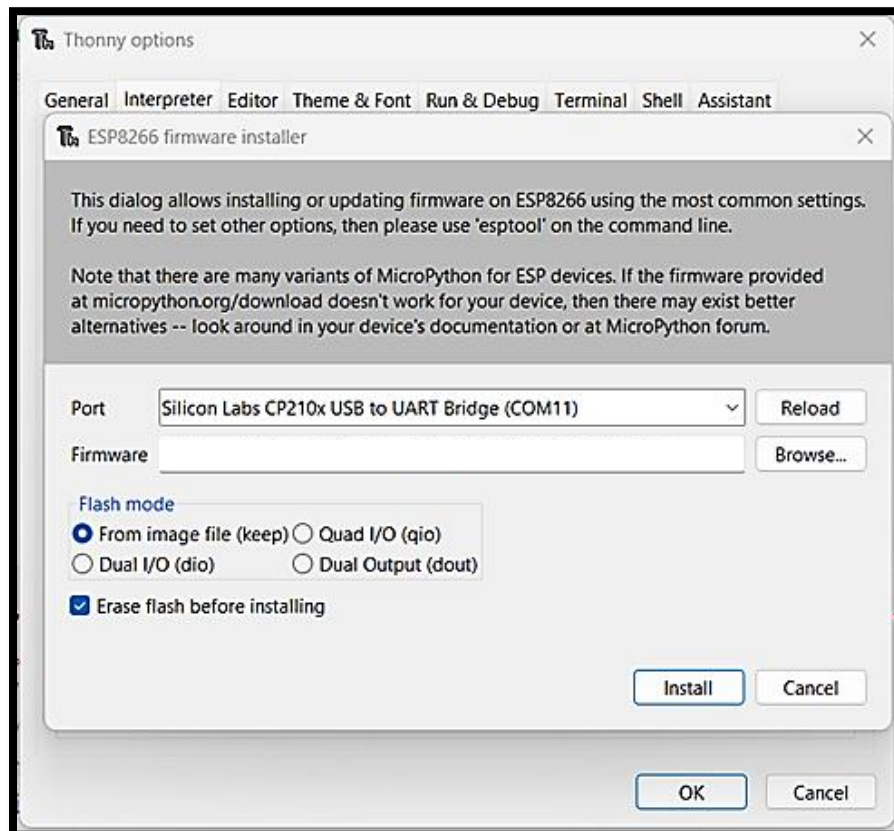
(نگه داشتن کلید flash و اتصال به سیستم)

سپس مجدد برنامه را باز کرده و وارد بخش configure interpreter شوید. بورد

مربوطه را انتخاب نمایید و سپس پورتهی که دستگاه به آن متصل است.



بر روی install or update micropython کلیک نمایید.
سپس در صفحه جدید ورودی های مربوط به پورت و فایل bin را وارد کرده و بر روی گزینه install کلیک نمایید.

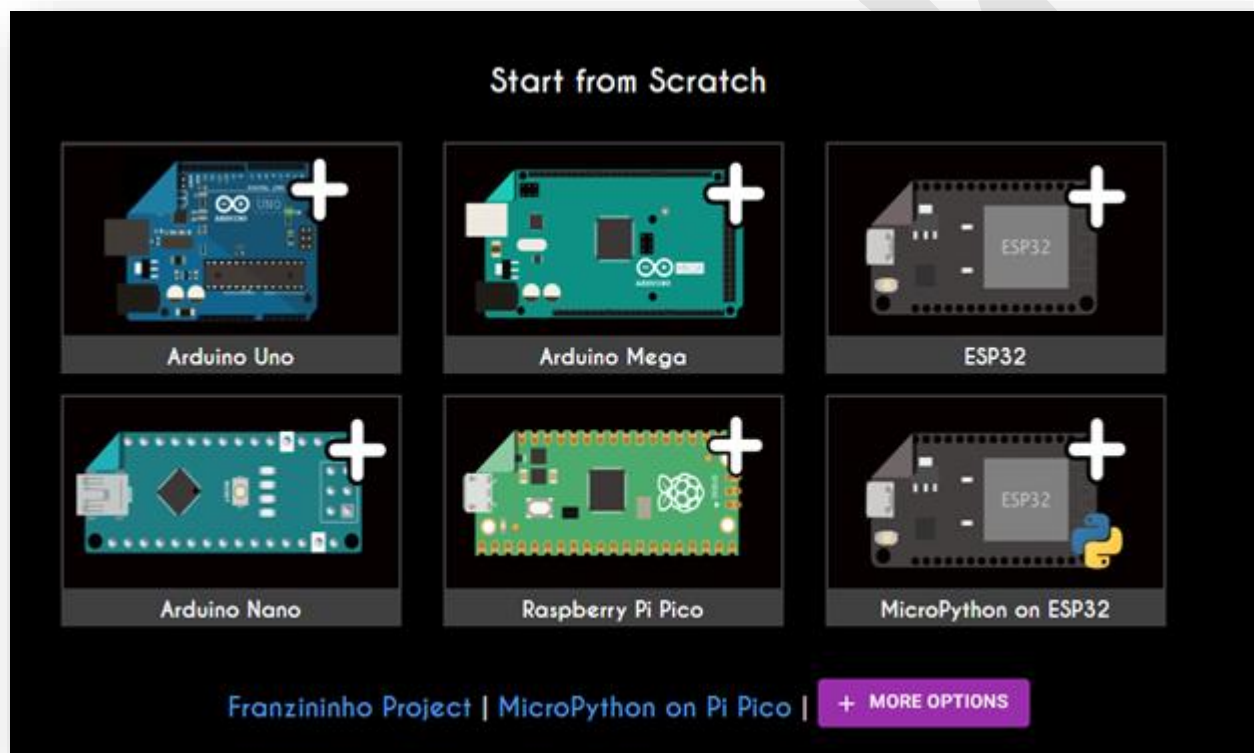


حالا روی گزینه Install می زنیم و تا پایان مراحل نصب صبر می کنیم.



۶ معرفی شبیه ساز Wokwi:

یکی از جذاب ترین شبیه ساز های آنلاین می باشد که انواع پروژه ها و خیلی ویژگی های خوبی را به همراه دارد و از زبان هایی مثل Arduino، C و micropython هم پشتیبانی می کند و حتی می توانید بورد های خاصی را هم برای انجام عملیات ها انتخاب کنید.



برای انتخاب Raspberry Pico یا ESP می توانیم از آدرس های زیر اقدام کنیم:

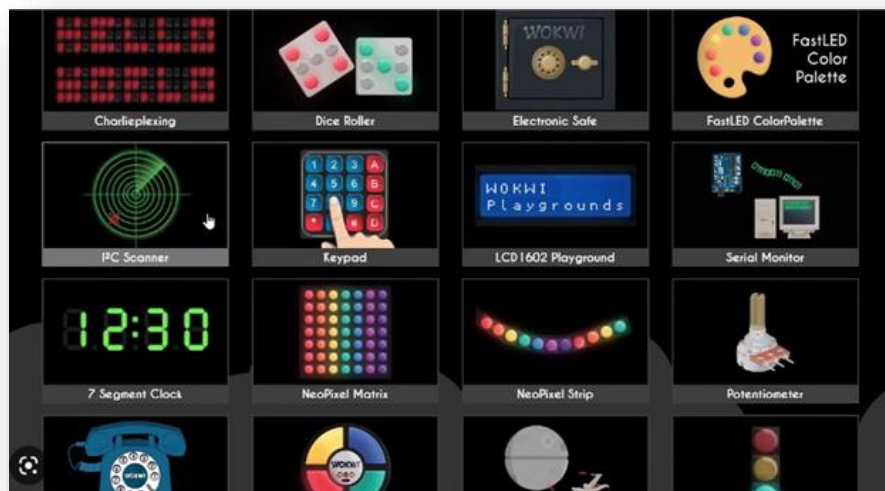
: Raspberry Pico

<https://wokwi.com/projects/new/micropython-pi-pico>

: ESP32

<https://wokwi.com/projects/new/micropython-esp32>

از مهمترین بخش های این شبیه ساز قابلیت کد نویسی هم زمان و اجرا به همراه استفاده از ماژول های متعدد است که به شما برای آمادگی کار با تجهیزات واقعی توانایی لازم انتقال می دهد .



۷ شروع کد نویسی

برای اینکه بتوانید با بورد خود ارتباط برقرار کنید و کد های خود را بر روی آن اجرا کنید دو راه وجود دارد:

(۱) استفاده از REPL

(۲) بارگذاری فایل

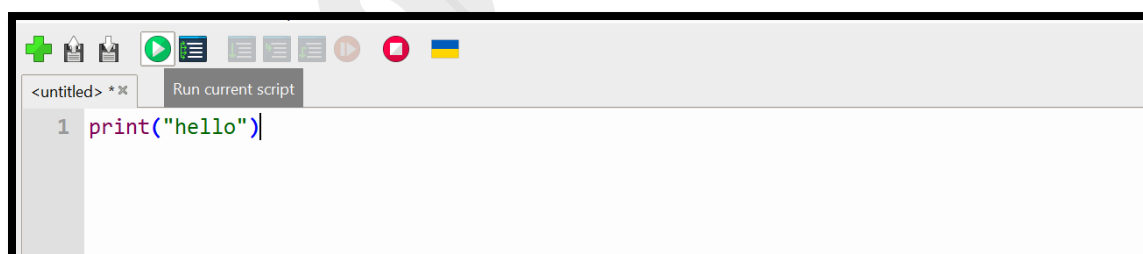
در صورتی که با پایتون آشنا باشید می دانید که در سیستم هم همین دو روش وجود دارد که شما می توانید با استفاده از terminal یا cmd دستورات پایتونی را خط به خط اجرا نمایید و یا اینکه به صورت منسجم با استفاده از یک فایل اجرا کنید.

در حالت استفاده از REPL می توانید دستورات خود را از طریق کنسول داخلی thonny اعمال کنید. مانند تصویر زیر.

```
Shell x
Unable to connect to COM5: port not found
Process ended with exit code 1.

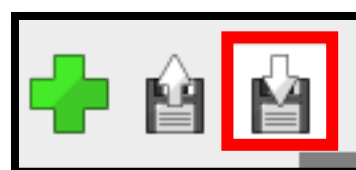
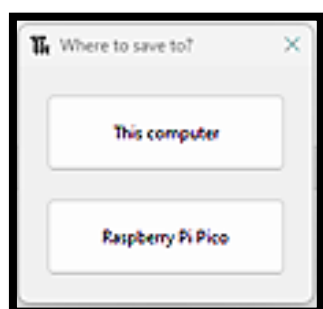
Python 3.11.4 (C:\Users\user\AppData\Local\Programs\Python\Python311\python.exe)
>>> print("hello")
hello
>>> |
```

و یا اینکه می توانید دستورات مربوطه را داخل بخش کد نویسی IDE وارد نمایید و سپس با فشردن کلید F5 و یا کلیک بر روی گزینه سبز رنگ PLAY آن را به اجرا در آورید.



۸ ذخیره کدها:

برای ذخیره کدها دو امکان داریم یکی ذخیره روی سیستم و دیگری ذخیره روی خود دستگاه IOT:



برای دسترسی به پین های روی برد نیاز به تعریف نوع عملکرد و همچنین بکار گیری آن ها است که در ابتدا می بایست ماژول مربوطه را به قطعه کد خود اضافه کنیم، که این کار با استفاده از ماژول **machine** صورت میگیرد. برای دسترسی به پین های برد کافیت که از این کتابخانه کلاس Pin ها را فراخوانی کنیم.

`from machine import Pin`

حال برای دسترسی به یک پین کافیت که یک آبجکت نگهدارنده از نوع عملکرد و شماره پین مربوطه ایجاد نماییم.

`led = Pin("LED", Pin.OUT)`

برای تغییر مقدار پین روش های مختلفی داریم:

```
led.on() # put the pin in HIGH state

led.high() # put the pin in HIGH state

led.value(1) # put the pin in HIGH state

led.off() # put the pin in LOW state

led.low() # put the pin in LOW state

led.value(0) # put the pin in LOW state
```

```
Pin.IN  
  
Pin.OUT  
  
Pin.OPEN_DRAIN  
  
Pin.ALT  
  
Pin.ALT_OPEN_DRAIN  
  
Pin.ANALOG
```

بر طبق داکيومنت های micropython پين ها می توانند وضعیت های زیر را داشته باشند.

لينک داکيومنت پين ها :

<https://docs.micropython.org/en/latest/library/machine.Pin.html>

برای مثال LED چشمک زن :

```
from machine import Pin
```

```
from time import sleep
```

```
led = Pin("LED",Pin.OUT)
```

```
while True:
```

```
    sleep(1)
```

```
    led.toggle()
```

دستور toggle فقط روی رزبری کار می دهد برای بقیه دستگاه ها از on و off استفاده می کنیم.

خواندن مقدار یک کلید : (به کمک value())

```
from machine import Pin
```

```
switch = Pin(15,Pin.IN)
```

```
print(switch.value())
```



برای اینکه بتوانند این خواندن پین را به صورت مداوم انجام دهید می توانید این عملکرد را در یک حلقه تکرار نمایید. و همچنین می توان با اضافه کردن یک منطق ساده به کنترل led روی برد نیز پرداخت. به نمونه کد زیر دقت نمایید.

```
from machine import Pin

from time import sleep

switch = Pin(15,Pin.IN)

led = Pin("LED",Pin.OUT)

while True:

    print(switch.value())

    if switch.value() == 0:

        led.on()

    else:

        led.off()

    sleep(0.1)
```

۹ استفاده از PWM:

کنترل پایه می تواند نه فقط خاموش و روشن بلکه بازه ای بین این وضعیت باشد، که نتیجه آن کنترل مقیاسی خواهد بود. موتور های DC و یا چراغ هایی که قابلیت Dim دارند می توانند از این طریق کنترل شوند. اما این کنترل بر اساس PWM صورت می گیرد که در ادامه به توضیح آن خواهیم پرداخت.

PWM چیست؟

مدولاسیون پهنای باند (PWM)، یک روش بسیار موثر برای بدست آوردن ولتاژ خروجی قابل کنترل و قابل تنظیم از ولتاژ ورودی ثابت یا متغیر می باشد. به عبارت دیگر PWM یک اصطلاح برای یک نوع از سیگنال دیجیتال می باشد که در مدارهای مختلف برای کنترل استفاده می شود. PWM تکنیکی است که به کمک آن می توانیم ولتاژ پایه های خروجی میکروکنترلر، و در نتیجه سرعت موتور یا سایر قطعات جانبی که به میکروکنترلر متصل می شود را کنترل کنیم. یک وسیله دیجیتال همانند میکروکنترلر فقط می تواند دو سطح کنترلی ۷۵ و ۷۰ و یا در شاخص کاری مورد ما بین ۰ تا ۳.۳ را تولید کند حال اگر خواستیم ولتاژ ۱.۵ ولت و یا ۳ ولت و یا هر ولتاژی بین ۰ و ۳.۳ ولت تولید کنید از PWM استفاده می کنیم. برای اینکه بتوانید یک پین را به این حالت ببرید ابتدا نیاز است که پین مورد نظر را تعریف و سپس به عنوان یک شیء از کلاس PWM آن را تبدیل به یک instance کنید. که این کار به روش زیر ممکن می شود.

```
from machine import Pin,PWM

led_pin = Pin(2,Pin.OUT)

pwm = PWM(led_pin)
```

دیگر تغییراتی که می خواهید به پین وارد کنید شامل ۰ و ۱ به تنهایی نخواهد بود و می تواند بر اساس بازه ای بین ۰,۶۵۵۳۵ قرار بگیرد. که این عدد به دلیل unit16 بودن بزرگ است و مقدار کنترل بیشتری به شما می دهد. اما در بعضی مورد ها ممکن است نیازی به استفاده از این عدد و unit16 نباشد و بازه عددی بین ۰ و ۱۰۲۳ قرار بگیرد.

همچنین می توانید فرکانس تغییرات را نیز با متد freq مبتنی بر instance ایجاد شده تغییر دهید. **Pwm.freq(1000)**

برای اینکه بتوانید duty عملکرد را تغییر دهید کافی است که از متد duty_u16 برای دادن مقدار وضعیت استفاده نمایید.

pwm.duty_u16(65535)

قطعه کد زیر نمونه ای از یک Fade را بر روی پین مشخص شده به شما نشان خواهد داد و با تغییر بازه های موجود شدت نور را بر اساس حلقه ای که در آن قرار گرفته است زیاد و یا کم خواهد کرد.

```
from machine import Pin,PWM

from time import sleep

led_pin = Pin(13,Pin.OUT)

pwm = PWM(led_pin)

pwm.freq(1000)

while True:

    for duty in range(65536):

        sleep(0.0001)

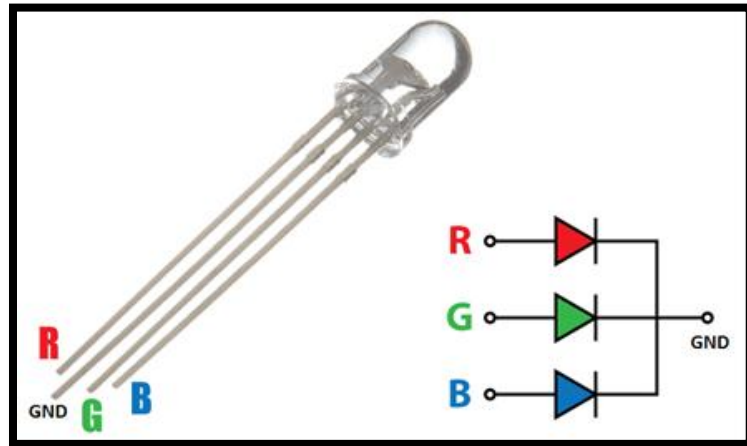
        pwm.duty_u16(duty)

    for duty in range(65546,-1,-1):

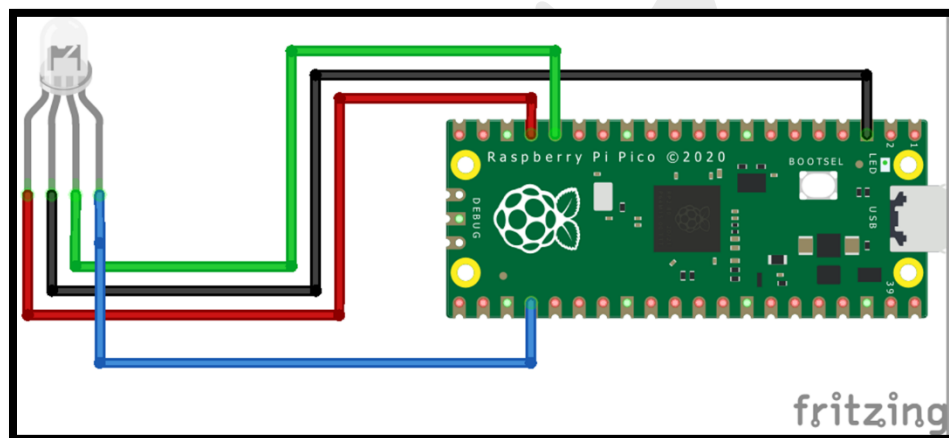
        sleep(0.0001)

        pwm.duty_u16(duty)
```

کاربرد دیگر PWM ساخت رنگ های مختلف با کمک LED RGB است.



شماتیک اتصالات :



برای کنترل led کافیست که هر پایه ای را که به هر رنگ متصل شده است را مجزا کنترل کنیم و در نهایت به تغییر میزان شدت نور رنگ های متنوع را ایجاد نماییم.

۱۰ راه اندازی ماژول های امنیتی:

(۱) ماژول رله:

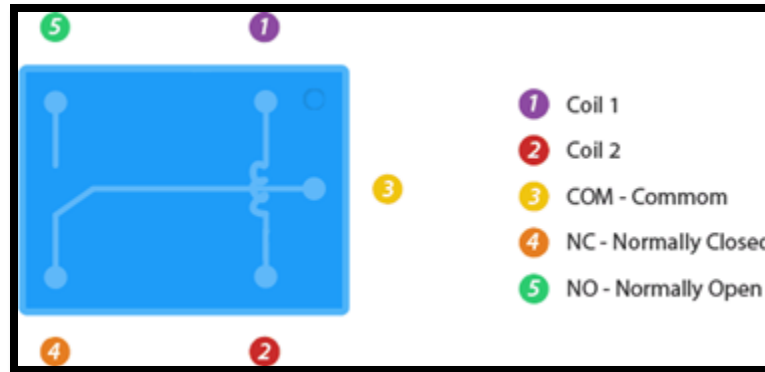
یک ماژول رله، یک دستگاه الکترونیکی است که به شما امکان کنترل مدارهای با ولتاژ و جریان بالا با استفاده از سیگنال های با ولتاژ و جریان پایین را می دهد. این دستگاه به طور رایج در پروژه های اینترنت اشیا (IoT) برای کنترل دستگاه هایی مانند چراغ، پنکه، موتور و دیگر وسایل بکار می رود.

ماژول رله شامل یک رله الکترومغناطیسی و یک مدار درایور است. مدار درایور برای کنترل رله با استفاده از اعمال ولتاژ به پیچ رله استفاده می شود. وقتی پیچ رله برقرار است، یک میدان مغناطیسی ایجاد می شود که مخاطب های سوئیچ را به هم می کشاند، مدار را بسته و به جریان اجازه می دهد که از طریق دستگاه کنترل شده جریان بگیرد.

در پروژه های IoT، سیگنال با ولتاژ و جریان پایین به طور معمول توسط یک میکروکنترلر یا یک حسگر تولید می شود. سیگنال سپس به مدار درایور ماژول رله فرستاده می شود که رله را فعال می کند و مدار با ولتاژ و جریان بالا را روشن یا خاموش می کند.

ماژول های رله در تنظیمات مختلفی مانند تک کاناله یا چند کاناله و با امتیازهای مختلف برای ولتاژ و جریان وجود دارند. انتخاب صحیح ماژول رله برای برنامه خود بسیار مهم است تا عملکرد ایمن و قابل اعتماد را تضمین کنید.

پایه های رله: رله ها معمولاً ۵ پایه دارند. ۳ پایه از این ۵ پایه، ترمینال هایی با ولتاژ بالا هستند. (COM, NO, NC) این پایه ها به دستگاه یا تجهیزاتی که قرار است شما آن را کنترل کنید، وصل می شوند.



در ماژول‌های رله، دو نوع ساختار به طور معمول کار می‌روند
Normally Closed (NC) و Normally Open (NO)

در ساختار **Normally Open**، پایه خروجی کنترل به پایه **NO** وصل می‌شود. در این حالت، وقتی که سیگنال کنترل به ماژول رله فرستاده می‌شود و رله فعال می‌شود، مدار بسته می‌شود و جریان از طریق دستگاه کنترل شده عبور می‌کند.

در ساختار **Normally Closed**، پایه خروجی کنترل به پایه **NC** وصل می‌شود. در این حالت، وقتی که سیگنال کنترل به ماژول رله فرستاده می‌شود و رله فعال می‌شود، مدار باز می‌شود و جریان از طریق دستگاه کنترل شده عبور نمی‌کند.

استفاده از ساختار **NO** یا **NC**، بسته به نیاز و نوع دستگاه کنترل شده قابل تنظیم است. بعضی از ماژول‌های رله دارای هر دو ساختار **NO** و **NC** هستند که به کاربر امکان کنترل دستگاه به دو صورت مختلف را می‌دهند.

نحوه اتصال :

پایه های دیجیتال شامل (GND,VCC) را به پایه های زمین و VCC میکرو و پایه دیجیتال IN1 را به یکی از پورت های دیجیتال برای ارسال دستور + یا ۱ برای قطع یا وصل کردن رله می‌فرستیم.

کد نویسی بخش رله:

```
import machine

import time

relay_pin = machine.Pin(13, machine.Pin.OUT)

# Turn on the relay for 1 second

relay_pin.on()

time.sleep(1)

# Turn off the relay

relay_pin.off()
```

در این کد، ابتدا پین ۱۳ به عنوان خروجی تعریف شده و به متغیر `relay\_pin` اختصاص داده می‌شود. سپس با فرستادن مقدار ۱ به `relay\_pin`، رله روشن شده و پس از گذشت ۱ ثانیه، با فرستادن مقدار ۰ به `relay\_pin`، رله خاموش می‌شود.

مهم است که مطمئن شوید که ولتاژ و جریان دستگاهی که قرار است توسط رله کنترل شود، با ولتاژ و جریان مجاز ماژول رله سازگار است و همچنین توجه داشته باشید که مقدار ولتاژ و جریان مناسب به رله ارسال شود.

۲) ماژول مگنت :

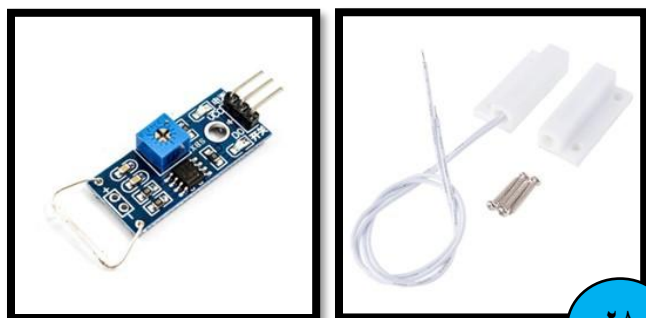
ماژول مگنت یک دستگاه الکترونیکی است که قابلیت تشخیص و اندازه‌گیری میدان مغناطیسی را داراست. این ماژول معمولاً شامل یک سنسور هال یا یک سوئیچ مغناطیسی است.

ماژول مگنت به دلیل قابلیت تشخیص میدان مغناطیسی، در بسیاری از کاربردهای مختلف مورد استفاده قرار می‌گیرد. برخی از کاربردهای این ماژول عبارتند از: دریافت اطلاعات از حرکت یک شیء، مانند درب یا پنجره، استفاده شود. با نصب یک مگنت بر روی درب یا پنجره و قرار دادن ماژول مگنت در نزدیکی آن، می‌توانید با تغییر میدان مغناطیسی، حرکت و بسته شدن درب یا پنجره را تشخیص دهید.

کنترل دوران موتورها: ماژول مگنت می‌تواند برای کنترل دوران موتورها مورد استفاده قرار گیرد. با قرار دادن مگنت‌هایی در موتور و نصب ماژول مگنت در نزدیکی آن، می‌توانید جهت و سرعت دوران موتور را کنترل کنید. اندازه‌گیری سرعت خودرو: ماژول مگنت می‌تواند برای اندازه‌گیری سرعت خودرو استفاده شود. با قرار دادن یک مگنت بر روی روتور چرخ خودرو و قرار دادن ماژول مگنت در نزدیکی آن، می‌توانید با تشخیص تعداد چرخش‌های روتور، سرعت خودرو را اندازه‌گیری کنید.

با توجه به کاربردهای مختلفی که در ماژول مگنت وجود دارد، این ماژول به عنوان یکی از قطعات الکترونیکی پرکاربرد در صنعت الکترونیک و IoT محسوب می‌شود.

نمونه ای از ماژول مگنت:



برای خواندن وضعیت باز یا بسته بودن درب با استفاده از ماژول مگنت که pull-up شده روی پین ۲، می‌توانید از کد زیر استفاده کنید:

```
import machine

import time

# Define the pin for the magnet sensor

magnet_pin = machine.Pin(2, machine.Pin.IN, machine.Pin.PULL_UP)

# Loop to read the state of the magnet sensor

while True:

    if magnet_pin.value() == 0:

        print("The door is closed.")

    else:

        print("The door is open.")

    time.sleep(0.1)
```

در این کد، ابتدا پین ۲ به عنوان ورودی تعریف شده و به متغیر `magnet\_pin` اختصاص داده می‌شود. همچنین، با استفاده از `machine.Pin.PULL\_UP`، پین به عنوان pull-up تنظیم می‌شود. سپس با استفاده از دستور `while True`، یک حلقه برای خواندن وضعیت ماژول مگنت تعریف شده است. در هر بار تکرار حلقه، با استفاده از `magnet\_pin.value()` وضعیت ماژول مگنت خوانده می‌شود و در صورتی که وضعیت ۰ باشد، به این معنی است که درب بسته است و در صورتی که وضعیت ۱ باشد، به این معنی است که درب باز است.

در این کد، با استفاده از `time.sleep(0.1)`، تاخیری به طول ۱/۰ ثانیه قرار داده شده است تا مقدار ماژول مگنت به طور پایدار خوانده شود. می‌توانید این مقدار را تغییر دهید تا مقدار دلخواه خود را برای تاخیر بین خواندن وضعیت ماژول مگنت قرار دهید.

۳) ماژول PIR:

ماژول PIR (Passive InfraRed) یک سنسور حرکت است که معمولاً برای تشخیص حرکت انسان‌ها و حیوانات در محیط استفاده می‌شود. این سنسور از تکنولوژی Infrared استفاده می‌کند و به دنبال تغییرات دمایی در محیط است. ماژول PIR در دو نوع تک پرتابل و دو پرتابل وجود دارد. در نوع تک پرتابل، سنسور حرکت در یک جهت خاص می‌تواند حرکت را تشخیص دهد، در حالی که در نوع دو پرتابل، سنسور حرکت در دو جهت مختلف می‌تواند حرکت را تشخیص دهد. ماژول PIR جزو قطعات الکترونیکی پرکاربرد در سیستم‌های امنیتی، خانه‌های هوشمند، سیستم‌های روشنایی خودکار و ... می‌باشد. برخی از کاربردهای این ماژول عبارتند از: سیستم‌های امنیتی: در سیستم‌های امنیتی، ماژول PIR می‌تواند برای تشخیص حرکت افراد ناخواسته و ناشناخته در محیط استفاده شود. با نصب ماژول PIR در نقاط استراتژیکی، می‌توانید هشدارهای لازم را در صورت تشخیص حرکت ناخواسته دریافت کنید. سیستم‌های خانه هوشمند: در سیستم‌های خانه هوشمند، ماژول PIR می‌تواند برای تشخیص حرکت افراد در داخل خانه استفاده شود. با نصب

ماژول PIR در نقاط استراتژیکی خانه، می‌توانید از حرکت افراد برای کنترل روشنایی، سیستم‌های گرمایش و سرمایش و ... استفاده کنید.

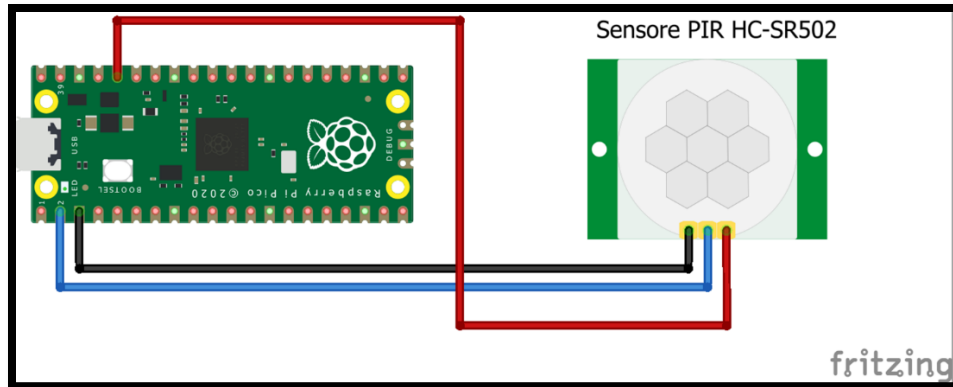
سیستم‌های روشنایی خودکار: در سیستم‌های روشنایی خودکار، ماژول PIR می‌تواند برای روشن و خاموش کردن نورها در محیط استفاده شود. با نصب ماژول PIR در نقاط استراتژیکی، می‌توانید هنگام تشخیص حرکت، نورهای محیط را روشن کنید و پس از توقف حرکت، نورها را خاموش کنید.

برای کار با ماژول PIR، این ماژول باید به منبع تغذیه DC وصل شده و خروجی آن، معمولاً به یکی از پین‌های ورودی میکروکنترلر وصل می‌شود.

هنگامی که ماژول PIR تغییری در دمای محیط تشخیص دهد، خروجی آن تغیر می‌کند و از طریق پین‌های ورودی میکروکنترلر، می‌توانید این تغیر را برای انجام عملیات مختلف، مانند روشن کردن نورها، اعلام هشدار و ... استفاده کنید. برای کار با ماژول PIR، باید مدار آن را بر اساس دیتاشیت سازنده و با توجه به میکروکنترلر مورد استفاده، طراحی کنید.



برای اتصال این سنسور هم پایه های VCC , GND به پایه های متناظر در میکرو متصل می شوند و پایه انتقال اطلاعات به یکی از پین های دیجیتال میکرو.



```
import machine
import time
# Define the PIR sensor pin
pir_pin = machine.Pin(2, machine.Pin.IN)

# Loop to read the state of the PIR sensor
while True:

    if pir_pin.value() == 1:

        print("Motion detected!")

    else:

        print("No motion detected.")

    time.sleep(0.1)
```

در این کد، ابتدا پین ۲ به عنوان ورودی تعریف شده و به متغیر `pir\_pin` اختصاص داده می‌شود. سپس با استفاده از دستور `while True`، یک حلقه برای خواندن وضعیت ماژول PIR تعریف شده است. در هر بار تکرار حلقه، با استفاده از `pir\_pin.value()`، وضعیت ماژول PIR خوانده می‌شود و در صورتی که وضعیت ۱ باشد، به این معنی است که حرکتی تشخیص داده شده و در صورتی که وضعیت ۰ باشد، به این معنی است که حرکتی تشخیص داده نشده است. در این کد، با استفاده از `time.sleep(0.1)`، تاخیری به طول ۱/۱۰ ثانیه قرار داده شده است تا مقدار ماژول PIR به طور پایدار خوانده شود. می‌توانید این مقدار را تغییر دهید تا مقدار دلخواه خود را برای تاخیر بین خواندن وضعیت ماژول PIR قرار دهید. در این کد، وقتی که حرکتی

تشخیص داده شود، پیام "Motion detected!" چاپ می‌شود و در غیر این صورت، پیام "No motion detected." چاپ می‌شود. شما می‌توانید به جای چاپ پیام، هرگونه عملیات دلخواه را برای تشخیص حرکت انجام دهید.

۴) ماژول التراسونیک:

اولتراسوند یا موج های فراصوتی، امواجی با فرکانس بالاتر از حد دسترسی شنوایی انسان (حدود ۲۰ کیلوهرتز) هستند. در اینترنت اشیا (IoT)، اولتراسوند می تواند به عنوان یک روش ارتباطی بین دستگاه های الکترونیکی استفاده شود. به عنوان مثال، در محیط هایی که امواج رادیویی ممکن است مشکل ساز باشند، مانند درون دیوارهای بتنی، اولتراسوند می تواند به عنوان یک راه حل جایگزین استفاده شود. این فناوری می تواند برای ارتباط بین دستگاه های IoT در محیط های صنعتی، مانند کارخانه ها، کاربردهای امنیتی، مانیتورینگ و کنترل پروسه های تولید و مصرف انرژی مورد استفاده قرار گیرد. همچنین، اولتراسوند می تواند برای تشخیص و کنترل اشیایی مانند درب های اتوماتیک و سیستم های امنیتی استفاده شود. به طور کلی، استفاده از اولتراسوند در IoT، مزایایی مانند پایداری بیشتر، قابلیت انتقال اطلاعات بیشتر و کارایی بالاتر را در بر خواهد داشت.

ماژول های اولتراسوندیک یا ultrasonic می توانند در بسیاری از کاربردهای IoT مورد استفاده قرار گیرند. برخی از استفاده های مهم آنها عبارتند از: سنسور اولتراسوندیک: برای اندازه گیری فاصله بین دو شی، مانند حضور یا عدم حضور یک شی در یک اتاق یا محیط خاص. سیستم های امنیتی: استفاده از ماژول های اولتراسوند برای تشخیص حرکت یا نفوذ در محیط هایی مانند ساختمان ها، خودروها و دستگاه های الکترونیکی. اتصالات بی سیم: با استفاده از فرکانس بالای اولتراسوند، می توان اطلاعات را از طریق امواج صوتی انتقال داد. این روش به عنوان یک راه حل برای اتصالات بی سیم در IoT استفاده می شود.

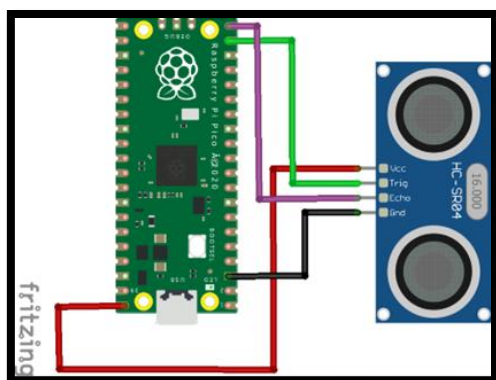
کنترل خودرو: ماژول های اولتراسوند می توانند برای کنترل حرکت خودروها در فضای باز و بسته استفاده شوند. به عنوان مثال، در سیستم های خودران، اولتراسوند می تواند برای تشخیص فاصله بین خودروها و جلوگیری از تصادفات استفاده شود.



نمونه ای از ماژول Ultrasonic:

نحوه اتصال به برد:

دو پایه VCC, GND باز هم به پایه های متناظر در میکرو متصل می شوند اما دو پایه trigger, echo برای ارسال موج و دریافت موج و محاسبه فاصله از طریق زمان رفتن و بازگشتن این موج هر دو به پین های دیجیتال متصل می شوند.



نمونه برنامه :

```
import machine

import time

# Define GPIO pins for the ultrasonic module
trigger_pin = machine.Pin(14, machine.Pin.OUT)

echo_pin = machine.Pin(15, machine.Pin.IN)

# Function to measure distance using the ultrasonic module
def measure_distance():
```

```

trigger_pin.low()

time.sleep_us(2)

trigger_pin.high()

time.sleep_us(10)

trigger_pin.low()

while echo_pin.value() == 0:

    signaloff = time.ticks_us()

while echo_pin.value() == 1:

    signalon = time.ticks_us()

# Calculate the duration of the Echo signal

timepassed = signalon - signaloff

# Calculate the distance based on the speed of sound

distance = (timepassed * 0.0343) / 2

return distance

# Measure the distance and print the result
while True:

    distance = measure_distance()

    print("Distance:", distance, "cm")

    time.sleep(1)

```

توضیح کدها :

در این برنامه ابتدا پین ۱۴ به عنوان خروجی برای ارسال موج و پین ۱۵ به عنوان ورودی برای دریافت موج استفاده شده است .

سپس در تابع `measure_distance` ابتدا `trigger` خاموش است و پس از ۲ میکرو ثانیه روشن می شود سپس پس از ۱۰ میکروثانیه دوباره خاموش شده و پالس ایجاد می کند.

در همین حین تا زمانی که `echo` چیزی دریافت نکرده زمان را در متغیر `signaloff` ذخیره می کنیم به محض دریافت صوت زمان دریافت را در `signalon` ذخیره می کنیم. سپس فاصله دو زمان را محاسبه و در فرمول زیر قرار می دهیم :

$$\text{distance} = (\text{timepassed} * 0.0343) / 2$$

اکنون به فاصله دسترسی داریم.

۱۱ راه اندازی ماژول های نظارتی (مانیتورینگ محیط)

a. ماژول DHT (اندازه گیری دما و رطوبت):

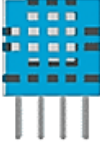
ماژول های DHT11 و DHT22 ، دو نوع سنسور رطوبت و دما هستند که با استفاده از آنها می توان دما و رطوبت محیط را اندازه گیری کرد. این دو سنسور قابلیت اتصال به میکروکنترلرها و بردهای الکترونیکی را دارند و از پروتکل دیجیتالی یکپارچه (Single Wire Digital Interface) برای ارتباط با میکروکنترلرها استفاده می کنند.

سنسور DHT11 دارای دقت پایین‌تری نسبت به DHT22 است و با دقت ۱ درصد رطوبت و دقت ۲ درصد دمای محیط را اندازه‌گیری می‌کند. این سنسور قابلیت اندازه‌گیری دمای محیط را در بازه ۵۰-۰ درجه سانتیگراد و رطوبت را در بازه ۹۰-۲۰ درصد دارد.

سنسور DHT22 دقت بیشتری نسبت به DHT11 دارد و با دقت ۰.۵ درصد رطوبت و دقت ۰.۲ درصد دمای محیط را اندازه‌گیری می‌کند. این سنسور قابلیت اندازه‌گیری دمای محیط را در بازه ۴۰- تا ۸۰ درجه سانتیگراد و رطوبت را در بازه ۱۰۰-۰ درصد دارد.

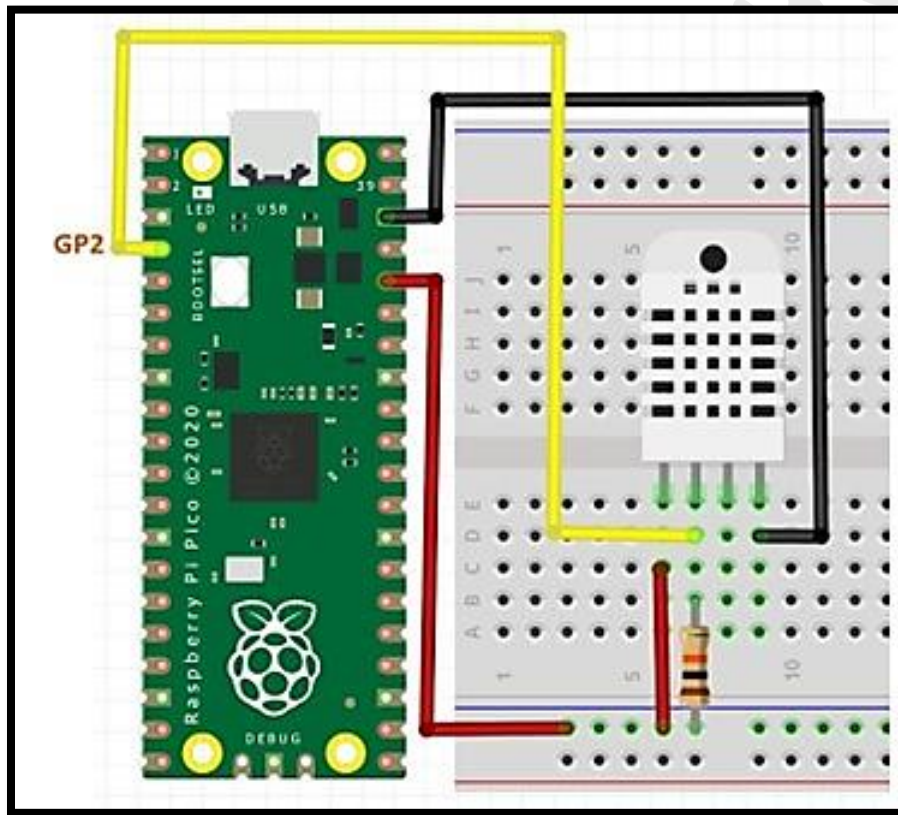
استفاده از این دو سنسور در IoT بسیار مفید است. به عنوان مثال، می‌توان از این سنسورها برای کنترل شرایط محیطی در یک اتاق، اندازه‌گیری دما و رطوبت در یک گلخانه، ایجاد سیستم خودکار کنترل دما و رطوبت در یک محیط ساختمانی و غیره استفاده کرد. با اتصال این سنسورها به یک برد IoT می‌توان به راحتی داده‌های اندازه‌گیری شده را به یک سیستم ابری ارسال و داده‌هایی مانند گراف تغییرات دما و رطوبت را بر روی یک پنل کاربری نمایش داد.

مقایسه این دو ماژول:

	DHT11	DHT22
		
Operating Voltage	3 to 5V	3 to 5V
Max Operating Current	2.5mA max	2.5mA max
Temperature Range	0-50°C / $\pm 2^{\circ}\text{C}$	-40 to 80°C / $\pm 0.5^{\circ}\text{C}$
Humidity Range	20-80% / 5%	0-100% / 2-5%
Sampling Rate	1 Hz (reading every second)	0.5 Hz (reading every 2 seconds)
Advantage	low cost	More Accurate

نحوه اتصال :

بعضی از این ماژول ها ۴ پایه و برخی ۳ پایه دارند ، برای اتصال به
بورد باید پایه های **VCC,GND** را به پایه های متناظر میکرو
متصل کرد و یکی از پایه ها که خروجی دما و رطوبت هست به
یکی از پایه های **ADC** میکرو متصل کرد.
اگر سنسور ۴ پایه داشت یکی از پایه ها بدون استفاده می ماند.



* میکروپایتون برای محاسبه دما و رطوبت از روی ولتاژ خروجی
خودش کتابخانه آماده دارد.

کد نویسی :

```
import machine
import dht
import time

# Define GPIO pin for DHT22 sensor
dht_pin = machine.Pin(2, machine.Pin.IN, machine.Pin.PULL_UP)

# Initialize DHT22 sensor object
d = dht.DHT22(machine.Pin(2))

# Function to read data from DHT22 sensor
def read_dht22():
    d.measure()
    return d.temperature(), d.humidity()

# Read data from DHT22 sensor and print to console
while True:
    temp, humidity = read_dht22()
    print("Temperature: ", temp, "°C")
    print("Humidity: ", humidity, "%")
    time.sleep(1)
```

از طریق `dht` می توانیم `DHT22` یا `DHT11` را نسبت به سنسور خود `import` کنیم.

ابتدا پین مربوط به `dht` را ورودی و `pull up` می کنیم. سپس همان پین را به کلاس `DHT22` یا `DHT11` می دهیم و درون متغیر `d` ذخیره می کنیم.

نهایتاً با دستور `d.measure()` دما و رطوبت اندازه گیری می شوند و از طریق دو دستور `d.temperature()` و `d.humidity()` در دسترس هستند.

b. سنسور فوتوسل:

سنسور فوتوسل یا LDR (Light Dependent Resistor) یک سنسور روشنایی است که می‌تواند تغییرات نوری در محیط را اندازه‌گیری کند. این سنسور با توجه به شدت نور محیط، مقاومت الکتریکی متفاوتی نشان می‌دهد. در واقع، با افزایش شدت نور، مقاومت الکتریکی سنسور کاهش می‌یابد و برعکس با کاهش شدت نور، مقاومت الکتریکی سنسور افزایش می‌یابد.

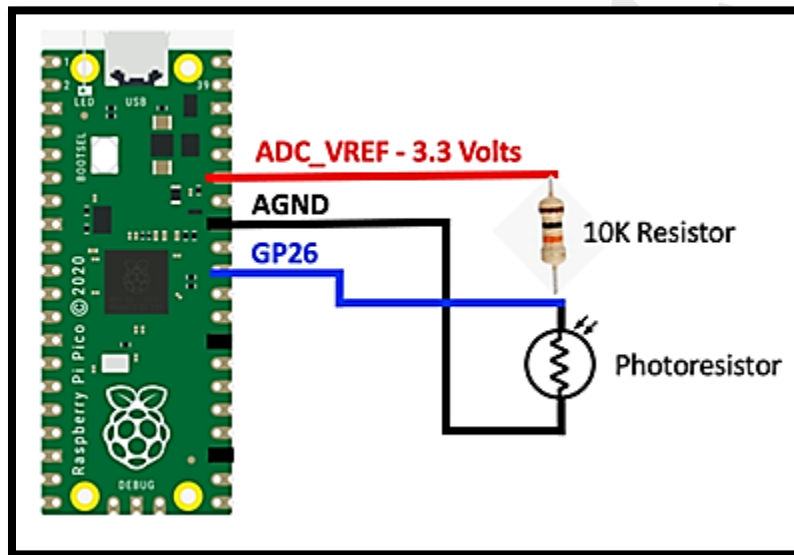
یکی از کاربردهای این سنسور در اینترنت اشیا، کنترل روشنایی است. با استفاده از سنسور فوتوسل و میکروکنترلر، می‌توان به راحتی مقدار نور محیط را اندازه‌گیری کرده و سپس با توجه به آن، دستگاه‌هایی مانند لامپ‌های هوشمند، کنترلرهای سیستم‌های خنک‌کننده و غیره را کنترل کرد. علاوه بر این، با اتصال سنسور فوتوسل به یک برد IoT، می‌توان داده‌ها را به یک سیستم ابری ارسال و از راه دور کنترل کرد. در کنترل وسایل نورپردازی نیز از این سنسور استفاده می‌شود. با استفاده از سنسور فوتوسل، می‌توان تغییرات شدت نور را اندازه‌گیری کرده و سپس به راحتی وسایل نورپردازی را کنترل کرد. در کل، سنسور فوتوسل یکی از سنسورهای مفید در اینترنت اشیا است که به راحتی می‌توان از آن در کنترل روشنایی و کنترل وسایل نورپردازی استفاده کرد.

نمونه ای از ماژول photocell:



نحوه اتصال:

این سنسور ۳ پایه دارد که دوتا برای تغذیه هستند و به VCC, GND میکرو متصل می شوند و پایه وسط برای انتقال اطلاعات به یکی از پایه های آنالوگ میکرو متصل می شود.



کد نویسی :

```
import machine
import time

# Define GPIO pin for photocell
photocell_pin = machine.ADC(26)

# Read data from photocell and print to console

while True:
    photocell_value = photocell_pin.read_u16()
    print("Photocell value: ", photocell_value)
    time.sleep(1)
```

ابتدا سنسور را به یکی از پایه های ADC متصل کرده و درون متغیر photocell_pin قرار می دهیم.

حالا از طریق **read_u16()** این مقدار خوانده و نمایش می دهیم.

c. ماژول MQX:

ماژول‌های سری MQX، یک سری سنسور گاز هستند که برای اندازه‌گیری میزان وجود گازهای خطرناک در محیط استفاده می‌شوند. این سنسورها با توجه به نوع گاز و شدت آن در محیط، مقدار ولتاژ خروجی خود را تغییر می‌دهند. با تبدیل این مقادیر ولتاژ به مقادیر دیجیتالی، می‌توان میزان وجود گاز را با دقت بالا اندازه‌گیری کرد.

این سنسورها در بسیاری از کاربردهای IoT مورد استفاده قرار می‌گیرند. به عنوان مثال، در سیستم‌های هوشمند خانه، این سنسورها می‌توانند به عنوان یکی از حسگرهای اصلی برای شناسایی وجود گازهای خطرناک مانند CO، CO₂، NH₃، NO₂ و غیره در محیط استفاده شوند. در صنعت نفت و گاز نیز این سنسورها به منظور شناسایی وجود گازهای خطرناک در محیط استفاده می‌شوند.

با اتصال سنسورهای MQX به یک برد IoT، می‌توان داده‌های اندازه‌گیری شده را به یک سیستم ابری ارسال و از راه دور کنترل کرد. به عنوان مثال، می‌توان این داده‌ها را به یک سیستم مانیتورینگ تلفیق شده با یادگیری عمیق ارسال کرد و با تحلیل این داده‌ها، میزان وجود گازهای خطرناک را شناسایی کرد و در صورت لزوم اقدامات مناسبی را به طور خودکار انجام داد.

در کل، سنسورهای MQX یکی از سنسورهای مهم در IoT هستند که می‌توانند در بسیاری از کاربردهای مرتبط با شناسایی وجود گازهای خطرناک مانند سیستم‌های هوشمند خانه، صنایع نفت و گاز و غیره مورد استفاده قرار گیرند. در زیر، مشخصات و ویژگی‌های مهم برخی از ماژول‌های سری MQ را بررسی می‌کنیم:



MQ-2:

-نوع گاز قابل شناسایی: متان، بوتان، پروپان، دود و غیره

-ولتاژ کاری: 5v DC

-محدوده اندازه گیری: ۳۰۰-۱۰۰۰۰ ppm

-خروجی: آنالوگ

-قطر سنسور: ۱۶ میلی متر

MQ-3:

-نوع گاز قابل شناسایی: الکل

-ولتاژ کاری: 5v DC

-محدوده اندازه گیری: ۲۵-۵۰۰ ppm

-خروجی: آنالوگ

-قطر سنسور: ۱۶ میلی متر

MQ-5:

-نوع گاز قابل شناسایی: گاز طبیعی، LPG، هگزان، پروپان و غیره

-ولتاژ کاری: 5v DC

-محدوده اندازه گیری: ۲۰۰-۱۰۰۰۰ ppm

-خروجی: آنالوگ

-قطر سنسور: ۳۲ میلی متر

:MQ-6

-نوع گاز قابل شناسایی: گاز طبیعی، LPG

-ولتاژ کاری: 5v DC

-محدوده اندازه گیری: ۲۰۰-۱۰۰۰۰ ppm

-خروجی: آنالوگ

-قطر سنسور: ۳۲ میلی متر

:MQ-7

-نوع گاز قابل شناسایی: CO

-ولتاژ کاری: 5v DC

-محدوده اندازه گیری: ۲۰-۲۰۰۰ ppm

-خروجی: آنالوگ

-قطر سنسور: ۳۲ میلی متر

:MQ-8

-نوع گاز قابل شناسایی: هیدروژن

-ولتاژ کاری: 5v DC

-محدوده اندازه گیری: ۱۰۰-۱۰۰۰۰ ppm

-خروجی: آنالوگ

-قطر سنسور: ۳۲ میلی متر

MQ-9:

-نوع گاز قابل شناسایی CO، CH₄، LPG

-ولتاژ کاری: 5v DC

-محدوده اندازه گیری CO: 10-1000ppm، CH₄، LPG: 100-10000ppm

-خروجی: آنالوگ

-قطر سنسور: ۳۲ میلی متر

MQ-135:

-نوع گاز قابل شناسایی CO₂، NH₃، NO_x، الکل، بنزن، دود و غیره

-ولتاژ کاری: 5v DC

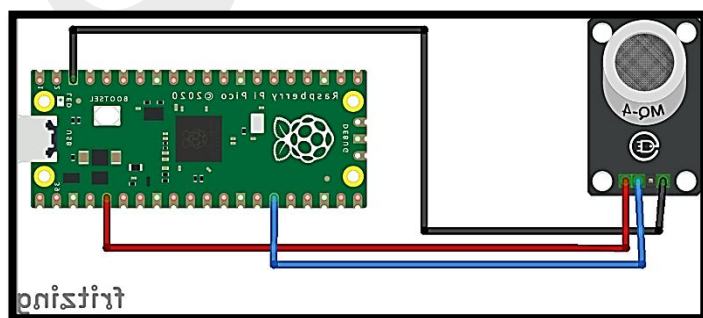
-محدوده اندازه گیری: ۱۰-۱۰۰۰ ppm

-خروجی: آنالوگ

-قطر سنسور: ۱۶ میلی متر

توجه داشته باشید که مقادیر ویژگی‌های مشخص شده برای هر سنسور، بسته به شرایط مختلف استفاده و نوع گازهایی که قرار است شناسایی شوند، ممکن است تغییر کند. همچنین، برای استفاده بهینه از این سنسورها، باید به دقت دستورالعمل‌های استفاده و کالیبراسیون هر سنسور را دنبال کرد.

نحوه اتصال :



کد نویسی :

```
import machine

import time

# Define GPIO pin for MQ-9 gas sensor

mq9_pin = machine.ADC(26)

# Read data from MQ-9 gas sensor and print to console

while True:

    mq9_value = mq9_pin.read_u16()

    print("MQ-9 value: ", mq9_value)

    time.sleep(1)
```

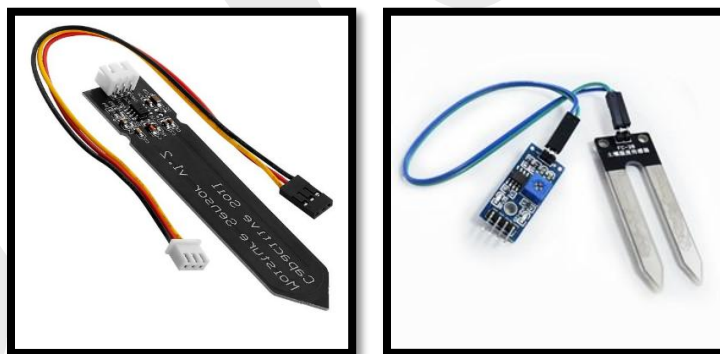
پایه خروجی سنسور را به یکی از پایه های ADC میکرو متصل می کنیم و شبیه سنسور فتوسل از طریق read_u16() مقدار ماژول mqx را می خوانیم .

d. ماژول رطوبت خاک:

ماژول اندازه گیری رطوبت خاک یک سنسور است که برای اندازه گیری میزان رطوبت خاک در باغچه، گلدان ها و یا هر محیط دیگری که نیاز به آبیاری دارد، استفاده می شود. این ماژول ها معمولاً شامل یک الکتروود و یک مدار الکترونیکی هستند که با اندازه گیری مقاومت الکتریکی خاک به عنوان یک شاخص برای میزان رطوبت خاک استفاده می شوند.

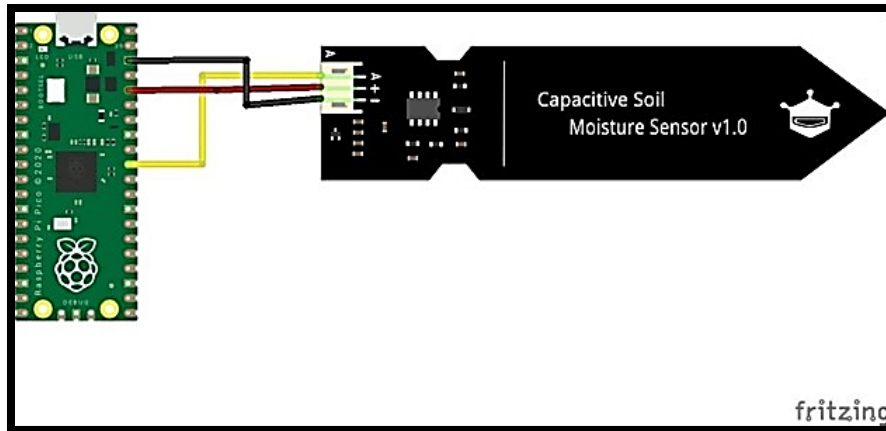
از آنجا که رطوبت خاک می‌تواند نقش مهمی در رشد و توسعه گیاهان داشته باشد، ماژول‌های اندازه‌گیری رطوبت خاک در بسیاری از کاربردهای مرتبط با کشاورزی و باغبانی استفاده می‌شوند. همچنین، در اینترنت اشیا، ماژول‌های اندازه‌گیری رطوبت خاک می‌توانند به عنوان یکی از عوامل اصلی در سیستم‌های هوشمند آبیاری استفاده شوند. برای مثال، با استفاده از این ماژول‌ها، می‌توان سیستمی را طراحی کرد که بر اساس مقدار رطوبت خاک، خودکار آبیاری گیاهان را کنترل کند و از هدر رفت آب جلوگیری کند. همچنین، این ماژول‌ها می‌توانند به عنوان یکی از عوامل کنترلی در سیستم‌های گلخانه‌ای استفاده شوند تا بهبود کیفیت و عملکرد محصولات کشاورزی را بهبود بخشند.

چند مدل از این ماژول:



نحوه اتصال:

نحوه اتصال دقیقاً شبیه فتوسل و MQX هست به طوری که دوتا از پایه‌های تغذیه و پایه دیگر برای تبادل اطلاعات است که به یکی از پین‌های ADC میکرو متصل می‌شود و با دستور `read_u16()` مقدار سنسور خوانده می‌شود.



کد نویسی :

```
import machine
import time

# Define GPIO pin for soil moisture sensor
soil_moisture_pin = machine.ADC(26)

# Read data from soil moisture sensor and print to console
while True:

    soil_moisture_value = soil_moisture_pin.read_u16()

    print("Soil moisture value: ", soil_moisture_value)

    time.sleep(1)
```

e. ماژول نئوپیکسل:

ماژول های Neopixel یا همان WS2812 یک نوع سنسور نوری دیجیتال هستند که قابلیت نمایش رنگ های مختلف را دارند.

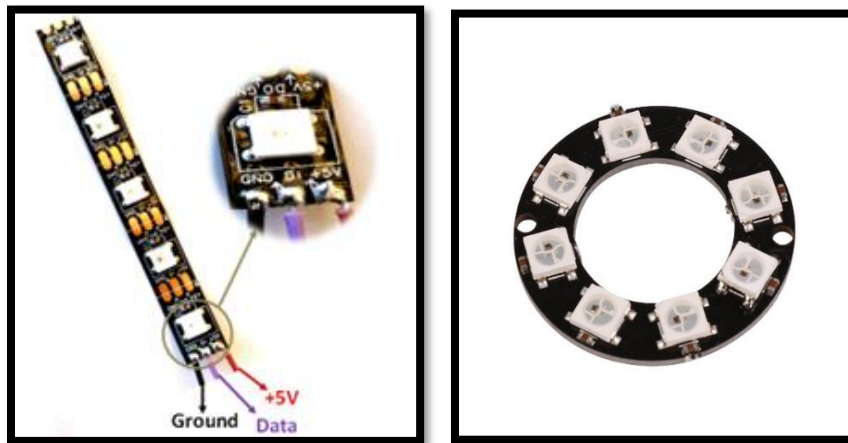
این ماژول‌ها شامل یک LED RGB و یک مدار الکترونیکی هستند که به کمک آن می‌توان رنگ LED را کنترل کرد.

Neopixel ها با استفاده از یک سیم مشترک برای تمام LED های یک رشته، به یکدیگر متصل هستند. هر LED با یک شناسه یکتا و برای تغییر رنگ آن LED است، کافی است که رنگ مورد نظر را به شناسه آن LED بفرستید.

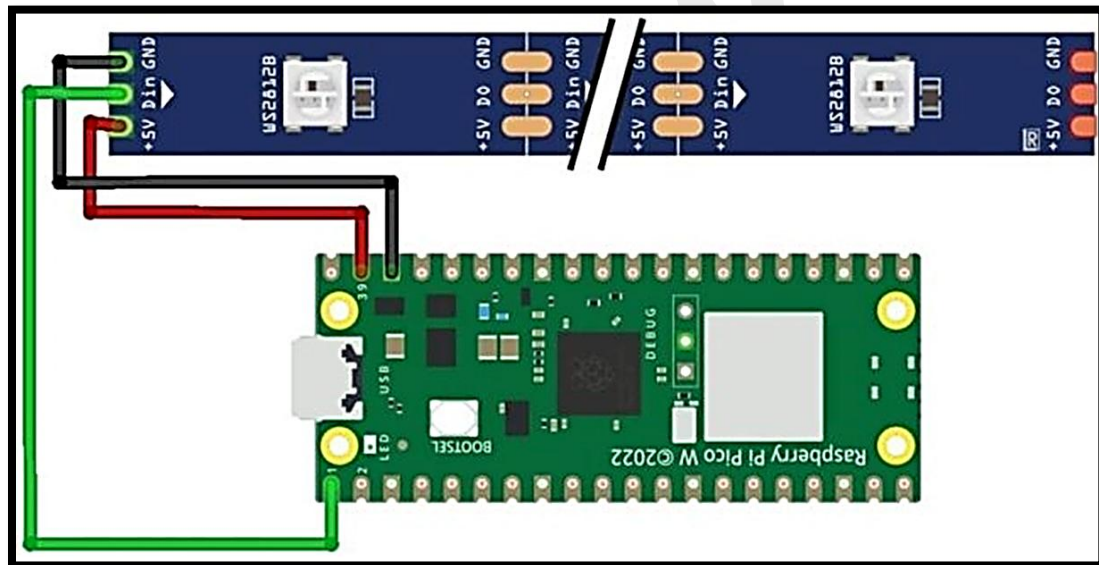
برای کار با ماژول‌های Neopixel در میکروکنترلرها، معمولاً از کتابخانه‌های مختلفی مانند FastLED و NeoPixel استفاده می‌شود. این کتابخانه‌ها، امکان کنترل رنگ و شدت نور LED ها را فراهم می‌کنند.

استفاده از ماژول‌های Neopixel در بسیاری از کاربردهای مرتبط با الکترونیک و رایانه‌ها مفید است. برای مثال، می‌توان از این ماژول‌ها برای ساخت نمایشگرهای LED، روشنایی اتاق‌ها، اشاره‌گرهای نوری، نمایشگرهای روشنایی و رنگی در سیستم‌های هوشمند، نمایشگرهای اعلان، نمایشگرهای شمارش معکوس و ... استفاده کرد. همچنین، این ماژول‌ها می‌توانند به عنوان یکی از اجزای کنترلی در ربات‌های هوشمند و خودروهای هوشمند استفاده شوند.

نمونه ای از ماژول های neopix:



نحوه اتصال:



نحوه کد نویسی :

```
import machine
import neopixel
import time

# Define the number of Neopixels
NUM_PIXELS = 8
```

```

# Define the GPIO pin for Neopixel data
NEOPIXEL_PIN = 1

# Setup the Neopixel object
np = neopixel.NeoPixel(machine.Pin(NEOPIXEL_PIN), NUM_PIXELS)

# Define some colors
RED = (255, 0, 0)

GREEN = (0, 255, 0)

BLUE = (0, 0, 255)

YELLOW = (255, 255, 0)

PURPLE = (255, 0, 255)

CYAN = (0, 255, 255)

WHITE = (255, 255, 255)

BLACK = (0, 0, 0)

# Set all pixels to red
np.fill(RED)
np.write()

# Wait for 1 second
time.sleep(1)

# Set all pixels to green
np.fill(GREEN)
np.write()

# Wait for 1 second
time.sleep(1)

# Set all pixels to blue
np.fill(BLUE)
np.write()

# Wait for 1 second
time.sleep(1)

```

```

# Set all pixels to yellow
np.fill(YELLOW)
np.write()

# Wait for 1 second
time.sleep(1)

# Set all pixels to purple
np.fill(PURPLE)
np.write()

# Wait for 1 second
time.sleep(1)

# Set all pixels to cyan
np.fill(CYAN)
np.write()

# Wait for 1 second
time.sleep(1)

# Set all pixels to white
np.fill(WHITE)
np.write()

# Wait for 1 second
time.sleep(1)

# Turn off all pixels
np.fill(BLACK)
np.write()

```

توضیحات :

ابتدا تعداد پیکسل ها و پایه آن ها را به شکل زیر به کتابخانه

neopixel پاس می دهیم :

`neopixel.NeoPixel(machine.Pin(NEOPIXEL_PIN), NUM_PIXELS)`

حالا می توانیم رنگ های مختلف RGB را تعریف کنیم :

```
# Define some colors
RED = (255, 0, 0)

GREEN = (0, 255, 0)

BLUE = (0, 0, 255)

YELLOW = (255, 255, 0)

PURPLE = (255, 0, 255)

CYAN = (0, 255, 255)

WHITE = (255, 255, 255)

BLACK = (0, 0, 0)
```

نهایتاً با دستورات زیر رنگ دلخواه روی پیکسل ها اعمال می شود:

np.fill(color)

np.write()

۱۲ اتصال به wifi:

این بخش را با یک مثال معرفی می کنیم .

برای ارسال اطلاعات به غیر از متصل شدن به wifi پروتکل ارسال اطلاعات نیز مهم هست که بهترین پروتکل ها برای iot ، پروتکل هایی هستند که به صورت realtime اطلاعات را ارسال می کنند مانند MQTT.

اما برای اینکه مثال قابل فهم تر باشد از یک پروتکل ساده و آشنا به نام HTTP استفاده می کنیم.

یعنی هربار اطلاعات را با درخواست post ارسال می کنیم به سرور و سرور با درخواست get آن ها را روی صفحه مانیتور می کند .

توضیحات کمی هم درباره سمت سرور می دهیم .

در این مثال مقدار دما و رطوبت را از طریق ماژول DHT22 و فاصله را با کمک ماژول التراسونیک می خوانیم .

خواندن اطلاعات دما و رطوبت :

```
import machine
import dht
# Define GPIO pin for DHT22 sensor
dht_pin = machine.Pin(15, machine.Pin.IN, machine.Pin.PULL_UP)
d = dht.DHT11(machine.Pin(15))
# Function to read data from DHT22 sensor
def read_dht11():
    d.measure()
    return d.temperature(), d.humidity()
```

خواندن فاصله:

```
import machine
# Define GPIO pins for the ultrasonic module
trigger_pin = machine.Pin(2, machine.Pin.OUT)
echo_pin = machine.Pin(18, machine.Pin.IN)
# Function to measure distance using the ultrasonic module

def measure_distance():
    trigger_pin.off()
    time.sleep_us(2)
    trigger_pin.on()
    time.sleep_us(10)
    trigger_pin.off()
    while echo_pin.value() == 0:
        signaloff = time.ticks_us()
    while echo_pin.value() == 1:
        signalon = time.ticks_us()
    # Calculate the duration of the Echo signal
    timepassed = signalon - signaloff
    # Calculate the distance based on the speed of sound
    distance = (timepassed * 0.0343) / 2
    return distance
```


حالا نیاز به اتصال به **wifi** همچنین امکان درخواست **http** داریم که از طریق کتابخانه **urequest** امکان پذیر است.

```
import urequests
import json
import network
import utime

wifi_sta = network.WLAN(network.STA_IF)
wifi_sta.active(True)
wifi_sta.connect("ssid", "password") # connect to the WiFi

while not wifi_sta.isconnected():
    # Optionally, print connection status
    print("Connecting...")
    # Delay for a short time
    utime.sleep(1)

print("Connection established!")
wlan_ip = wifi_sta.ifconfig()

# Print the device's IP address
print("IP Address:", wlan_ip)

while True:
    temp, humidity = read_dht11()
    distance = measure_distance()

    sensor_data = {
        "temperature": str(temp),
        "humidity": str(humidity),
        "gas": str(distance)
    }

    try:
        response =
urequests.post("http://192.168.1.144:8000/send/sensor/value",
data=json.dumps(sensor_data))
        print(response.text)
    except OSError as e:
        print(f"An error occurred while sending the request: {e}")
```

```
time.sleep(1)
```

توضیح بخش به بخش :

```
wifi_sta = network.WLAN(network.STA_IF)
wifi_sta.active(True)
wifi_sta.connect("ssid", "password") # connect to the WiFi

while not wifi_sta.isconnected():
    # Optionally, print connection status
    print("Connecting...")
    # Delay for a short time
    utime.sleep(1)

print("Connection established!")
wlan_ip = wifi_sta.ifconfig()

# Print the device's IP address
print("IP Address:", wlan_ip)
```

در این بخش ابتدا سعی می کنیم به ssid و password وایفای خود متصل شویم .

با حلقه while تا زمانی که wifi متصل نشده عبارت در حال اتصال(connecting...) را پرینت می کنیم.

زمان متصل شدن آدرس ip ها وایفای را نمایش می دهیم.

```
while True:
    temp, humidity = read_dht11()
    distance = measure_distance()

    sensor_data = {
        "temperature": str(temp),
        "humidity": str(humidity),
        "gas": str(distance)
    }
```

در این بخش اطلاعات دما و رطوبت و فاصله را از توابعی که قبلا نوشتیم می خوانیم و داخل یک **دیکشنری پایتونی** قرار می دهیم.

```
try:
    response =
urequests.post("http://192.168.1.144:8000/send/sensor/value",
data=json.dumps(sensor_data))
    print(response.text)
except OSError as e:
    print(f"An error occurred while sending the request: {e}")
```

در بخش آخر اطلاعات را به ip که به آن متصل شدیم به صورت post ارسال می کنیم فقط قبل از ارسال با دستور **json.dumps(sensor_data)** دیکشنری پایتونی را به json برای قابل فهم بودن برای سرور تبدیل می کنیم.

با کمک **try,except** اگر اطلاعات ارسال شد پاسخ گرفته شده از سرور را چاپ می کنیم در غیر این صورت خطا را چاپ می کنیم.

حالا کمی درباره سمت سرور توضیح دهیم:

اطلاعات می توانند به یک فضای ابری ارسال شوند یا روی یک پایگاه داده ذخیره شوند ، چون حجم اطلاعات زیاد است مثلا در هر ثانیه دما ذخیره می شود بهتر است پایگاه داده غیر sql (غیر رابطه ای) استفاده کنیم.

چون در بیشتر پروژه های iot هوش مصنوعی هم دخیل هست و بیشتر با کمک python در حال توسعه هستند تکنولوژی سمت سرور باید سبک و بر مبنای پایتون باشد برای پروژه های قوی از Fast Api برای پروژه های کمی ضعیف تر از Flask استفاده می کنیم. البته Django هم کاربرد دارد اما کمی سنگین است و برای ناهمزمان کار کردن نیاز به تنظیمات خاصی دارد.

در این پروژه من با Django مثال می زنم چون پروتکل ما real time نیست.

اگر بخواهیم پروژه روی لوکال ارتباط بگیرد و صرفاً از طریق آدرس ip مودم، میکرو و سرور با هم در ارتباط باشند و اطلاعات به خارج از حالت داخلی ارسال نشود ابتدا باید از آدرس ip مودم اطلاع پیدا کنیم.

برای این کار داخل windows، cmd را باز کرده و دستور ip config را می زنیم. داخل Linux از if config استفاده می کنیم.

```
Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix  . : 
IPv4 Address. . . . . : 192.168.1.144
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.1.1
```

حالا سرور جنگو را باید با دستور **python manage.py runserver 0.0.0.0:8000** ران کنیم تا بتواند از ip لوکال بهره ببرد.

حالا از طریق <http://ip:8000> به سرور خود دسترسی داریم.

جنگو یک فریم ورک پیشرفته هست و توضیح آن باعث دور شدن از موضوع اصلی می شود اما لینک مستندات جنگو در قسمت منابع قرار گرفته است.

۱۳ منابع:

[سایت رسمی میکروپایتون](#)

[سایت شبیه ساز WokWi](#)

[سایت رسمی رزبری پای](#)

[سایت رسمی Arduino برای اطلاعات node mcu](#)

[سایت رسمی thonny](#)

تمامی منابع لینک داده شده اند با کلیک روی هر کدام در تلفن همراه وارد سایت می شوید.(برای ورود با کامپیوتر باید دکمه CTRL را نگه دارید و سپس کلیک کنید.)